

Catatan Kuliah

IF3170 Artificial Intelligence

Semester I 2025/2026

6 Juli 2025

IF '23

Razi Rachman Widyadhana - @zirachw

Muhammad Fathur Rizky - @fathurwithyou

Apabila terdapat kesalahan pada file, silakan kontak penulis :)

Dilarang membuka dan membawa berkas ini saat ujian berlangsung

This page intentionally left blank

Daftar Isi

0	Preliminary	5
0.1	Dosen Pengampu	6
0.2	Workload	6
0.3	Important Dates	6
0.4	Aturan Perkuliahan	6
0.5	Resources	6
1	Introduction to AI	7
1.1	AI in a Nutshell	8
1.2	4 Approaches in AI Definition	8
1.3	Acting Humanly	9
1.4	Thinking Humanly	9
1.5	Acting Rationally	10
1.6	Thinking Rationally	10
1.7	Latihan	11
2	Agents in AI	12
2.1	What is an Agent?	13
2.2	Agent Model (Studi Kasus)	14
2.3	Rational Agent	15
2.4	PEAS	16
2.5	Task Environment	17
2.6	Agent Structure & Types	18
2.7	Agent Level (Studi Kasus)	21
3	Local Search	23
3.1	Classical Search Review	24
3.2	Local Search	24
3.3	Landscape Ruang State pada Local Search	26
3.4	State	26
3.5	Successor dan Neighbor	27
3.6	Hill Climb	27
3.6.1	Basic Hill Climb	27
3.6.2	Sideways	29
3.6.3	Stochastic	30
3.6.4	Random Restart	30
3.7	Simulated Annealing	30
3.8	Genetic Algorithm	31
3.8.1	Individu	32
3.8.2	Populasi	33
3.8.3	Selection	33
3.8.4	Cross Over & Mutation	34

4	Adversarial Search	35
4.1	Minimax Algorithm	36
4.2	Alpha-Beta Pruning	38
4.2.1	Mekanisme Kerja	38
4.2.2	Keunggulan dan Efisiensi	39
5	Constraint Satisfaction Problem	40
5.1	What is CSP?	41
5.2	Visualisasi CSP	41
5.3	Variasi dari Formulasi CSP	42
5.4	Constraint Propagation & Inference	43
5.5	Improving Backtracking Efficiency	44
5.6	Interleaving Search	45
5.7	Local Search for CSP	46
6	Knowledge-based System	47
6.1	What is KBS?	48
6.2	KBS: Why?	49
6.3	KBS vs Conventional Program	49
6.4	Knowledge-based Agent	50
6.5	Knowledge Representation (Short Review)	51
6.6	KBS Architecture	52
6.7	Knowledge Representation (Cont.)	55
6.8	Knowledge Engineering	56
7	Rule-based System	57
7.1	RBS: What and Why?	58
7.2	Rule: Logical Implication	58
7.3	Arsitektur RBS	59
7.4	Metode Inferensi (Penalaran)	59
7.5	Forward Chaining	60
7.6	Backward Chaining	63
7.7	Advantages of RBS	65
8	Learning	66
8.1	Introduction to Learning	67
8.2	Knowledge-based Agent with Learning	67
8.3	Learning: Why?	68
8.4	Arsitektur KBA with Learning	68
8.5	Design of A Learning Element	69
8.6	Learning Type based on Feedback	69
9	Supervised Learning	70
9.1	What is Supervised Learning?	71
9.2	Classification vs Regression	72
9.3	Overfitting vs. Generalization	72
9.4	Supervised Learning Pipeline	73

Bab 0

Preliminary

0.1	Dosen Pengampu	6
0.2	Workload	6
0.3	Important Dates	6
0.4	Aturan Perkuliahan	6
0.5	Resources	6

Mata kuliah “Inteligensi Buatan” merupakan salah satu mata kuliah wajib prodi di Program Studi Teknik Informatika (S1) STEI ITB.

Mata kuliah ini berada dalam lingkup lab AI (Artificial Intelligence) dengan KK Informatika.

Memiliki beban 4 SKS yang terbagi menjadi 4 jam tatap muka di kelas, 4 jam mengerjakan berbagai tugas, dan 4 jam belajar mandiri. Selain itu, terdapat prerequisite LogKom, Stima, TBFO, dan ProbStat untuk Mata Kuliah ini.

0.1 Dosen Pengampu

- K1: Dr. Nur Ulfa Maulidevi, S.T, M.Sc.
- K2: Dr. Fariska Zakhralativa Ruskanda, S.T., M.T.
- K3: Dr. Eng. Ayu Purwarianti, S.T, M.T.

0.2 Workload

- Latihan Edunex
- 3 Praktikum
- 2 Tugas kecil, 2 Tugas Besar
- Belajar Mandiri (2 Kuis, UTS, UAS)

0.3 Important Dates

- Kuis 1 (Minggu ke-4): 22 September 2025
- UTS (Minggu ke-8)
- Prak 1 (Minggu ke-9): 29 Oktober 2025
- Kuis 2 (Minggu ke-12): 10 November 2025
- Prak 1 (Minggu ke-9): 24 November 2025
- UAS (Minggu ke-16)

0.4 Aturan Perkuliahan

- Syarat lulus IF3170: nilai setiap komponen > 0
- Segala bentuk kecurangan akademik akan mengakibatkan nilai E, bekerja sama antar kelompok berbeda, mengutip dari sumber lain tapi tidak disertakan acuannya (Google, ChatGPT, dll).
- Susulan Kuis, UTS, UAS hanya untuk yang dirawat di rumah sakit/terkena musibah besar.

0.5 Resources

Stuart J Russell & Peter Norvig, Artificial Intelligence: A Modern Approach, 4th Edition, Prentice-Hall International, Inc, 2022, Textbook

Bab 1

Introduction to AI

1.1	AI in a Nutshell	8
1.2	4 Approaches in AI Definition	8
1.3	Acting Humanly	9
1.4	Thinking Humanly	9
1.5	Acting Rationally	10
1.6	Thinking Rationally	10
1.7	Latihan	11

1.1 AI in a Nutshell

Is this an AI?



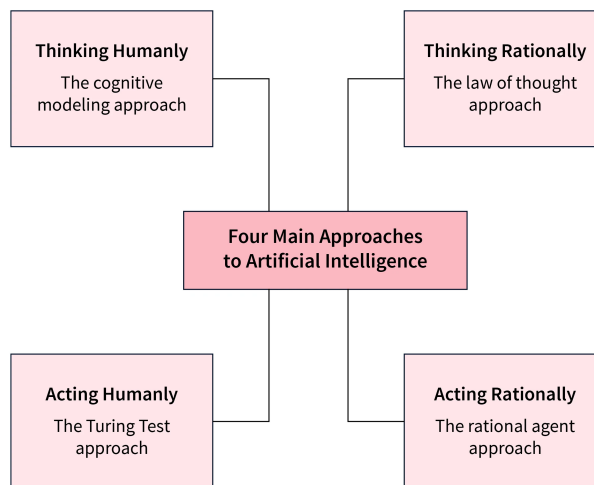
Gambar 1.1: Calculator, obviously

Try to answer these (yes/no) - Quiz IF3170 2024/2025:

1. Lampu yang menyala otomatis ketika ada orang di ruangan yang keberadaannya ditangkap oleh sensor pada lampu tsb
2. Aplikasi penjadwalan kuliah yang menggunakan algoritma pencarian
3. Robot pengantar makanan ke ruangan pasien yang dikendalikan dengan remote control
4. Aplikasi akuntansi yang menggunakan beragam fungsi matematika deterministik dan pelaporan otomatis
5. Aplikasi yang memilah barang catat berdasar gambar yang ditangkap kamera

1.2 4 Approaches in AI Definition

By Russel and Norvig



Gambar 1.2: 4 Approach dalam definisi AI

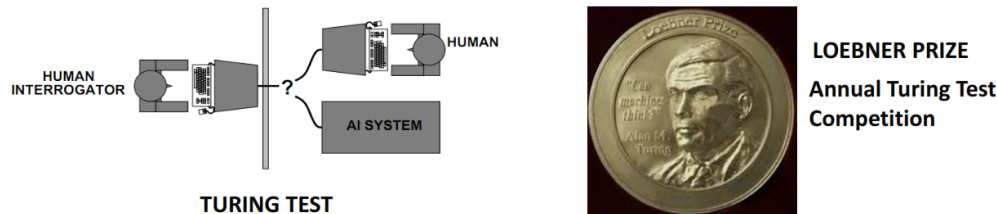
1.3 Acting Humanly

Konsep: The Turing Test Approach

"The art of creating machines that perform functions that require intelligence when performed by people (Kurzweil, 1990)

The study of how to make computers do things at which, at the moment, people are better (Rich and Knight, 1991)

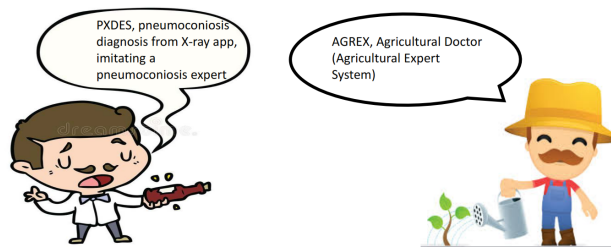
Artinya apa bang messi? Intinya, berfokus pada kemampuan *machines* untuk *mimic* seperti *human* pada umumnya. Bahkan, bisa aja ga bisa bedain ini yang *peragain* apakah *machine* atau *human*.



Gambar 1.3: Turing Test

Turing Test melibatkan seorang juri manusia yang berinteraksi dalam percakapan bahasa alami dengan sebuah mesin dan seorang manusia, **tanpa mengetahui mana di antara keduanya**. Jika mesin berhasil meyakinkan sang juri bahwa ia adalah manusia, maka mesin tersebut dianggap telah lulus Turing Test. Dengan kata lain, merupakan AI dengan pendekatan ***Acting Humanly***

Jadi, contohnya apa? Dari *turing test*, istilah yang sekarang ramai menjadi *chatbot*.



Gambar 1.4: Contoh lain dari AI dengan pendekatan *Acting Humanly* selain *chatbot*

1.4 Thinking Humanly

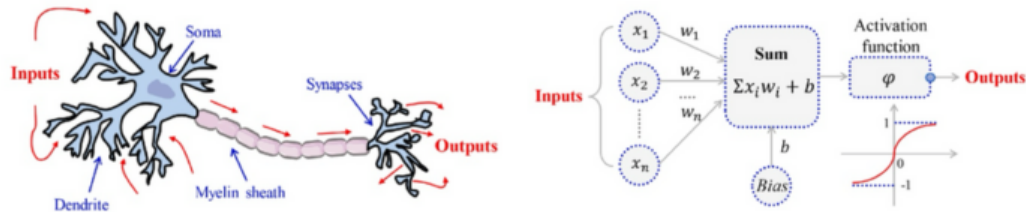
Konsep: The Cognitive Approach

[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ... (Bellman, 1978)

The exciting new effort to make computers think ... machines with minds, in the full and literal sense (Haugeland, 1985)

Pendekatan ini berkaitan dengan proses kognitif internal manusia. Dengan kata lain, bagaimana **manusia berpikir** dan bagaimana kita dapat merepresentasikan proses tersebut **dalam komputer**.

Contohnya? Model *artificial neural network* terinspirasi oleh *neural networks* dalam otak manusia. Meskipun model-model ini tidak sepenuhnya mencerminkan cara kerja otak, mereka berusaha meniru bagaimana *neuron* dalam otak manusia dapat memproses informasi.



Gambar 1.5: (a) Biological Neuron, (b) Artificial Neuron

1.5 Acting Rationally

Konsep

Computational intelligence is the study of the design of intelligent agents (Poole et al., 1998)

AI...is concerned with intelligent behavior in artifacts (Nilsson, 1998)

Berfokus pada hasil akhir dari tindakan AI, yaitu bertindak dengan **cara yang paling optimal atau efisien** untuk mencapai suatu tujuan.

Salah satunya adalah *Pathfinding algorithms* dalam permainan catur. Meskipun mereka mungkin tidak memikirkan permainan dengan cara yang sama seperti manusia, mereka bertujuan untuk membuat langkah terbaik berdasarkan posisi saat ini dan kemungkinan langkah di masa depan.



Gambar 1.6: Contoh AI dengan pendekatan *Acting Rationally*

1.6 Thinking Rationally

Konsep

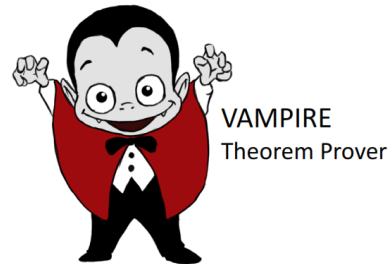
The study of mental faculties through the use of computational models (Charniak and McDermott, 1985)

The study of the computations that make it possible to perceive, reason, and act (Winston, 1992)

Pendekatan ini berkaitan dengan berpikir **secara logis** dan membuat keputusan berdasarkan penalaran. Dengan kata lain, **menggunakan logika** dalam pengambilan keputusan.

Gimana sih bentuknya? seperti **Rule-based systems** yang menggunakan **formal logic** untuk membuat keputusan. Misalnya, expert systems dalam bidang medis yang menganalisis gejala dan data pasien untuk memberikan diagnosis atau rekomendasi perawatan.

Ataupun *theorem solver*, Kenapa?



Gambar 1.7: Contoh AI dengan pendekatan *Acting Rationally*

1.7 Latihan

Berdasarkan empat pendekatan AI, tentukan pendekatan yang digunakan pada aplikasi/teknologi berikut, ataukah aplikasi tersebut tidak menggunakan pendekatan AI.

1. NuPIC, platform perangkat lunak yang berdasarkan pada model struktur dan operasi pada *neocortex* (bagian pada otak manusia)
2. PXDES, aplikasi yang melakukan diagnosis X-ray layaknya seorang pakar melakukan diagnosis, untuk penentuan *pneumoconiosis* (penyakit paru-paru yang disebabkan oleh penghisapan debu)
3. Pc-Nqthm, aplikasi 'proof-checker' yang berlandaskan pada teori automated reasoning, berkaitan dengan aturan formal logika.
4. AceMoney, aplikasi yang membantu mengorganisasikan dan mengatur keuangan individu (mencatat pemasukan dan pengeluaran)
5. Vampire, *automatic theorem prover* untuk *first order logic* yang menjuarai 11 kali *world cup in theorem prover* sejak 1999
6. Robot melakukan eksplorasi pada suatu lingkungan yang belum pernah dikenali sebelumnya. Robot tersebut bekerja untuk sampai pada lokasi tertentu yang diinginkan oleh pemiliknya dari posisi awal mereka diletakkan
7. Logic Problem Solver, aplikasi yang dapat membantu menyelesaikan persoalan logika yang ada di buku atau majalah logic puzzle
8. AGREX, aplikasi sistem pakar yang membantu petani/pembisnis agrikultur dengan memberikan saran yang benar pada saat yang tepat mengenai penjadwalan irigasi, diagnosis penyakit padi, pemupukan, dan perlindungan tanaman
9. Vitamin D, perangkat lunak yang digunakan untuk mendeteksi manusia atau objek bergerak pada video streams. Aplikasi ini memanfaatkan teknologi yang memodelkan *neocortex* (bagian dari otak manusia yang bertanggung jawab untuk high level perception)

Bab 2

Agents in AI

2.1	What is an Agent?	13
2.2	Agent Model (Studi Kasus)	14
2.3	Rational Agent	15
2.4	PEAS	16
2.5	Task Environment	17
2.6	Agent Structure & Types	18
2.7	Agent Level (Studi Kasus)	21

2.1 What is an Agent?

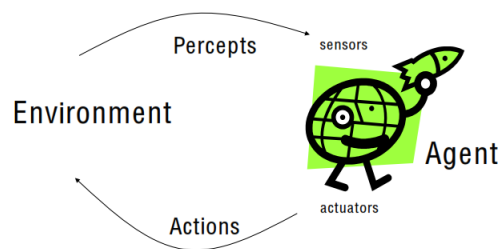
Definisi 2.1

Segala sesuatu yang dapat dipandang sebagai **persepsi** terhadap *environment* melalui *sensors* dan **bertindak** terhadap *environment* tersebut melalui *actuators*.

Dapat berupa:

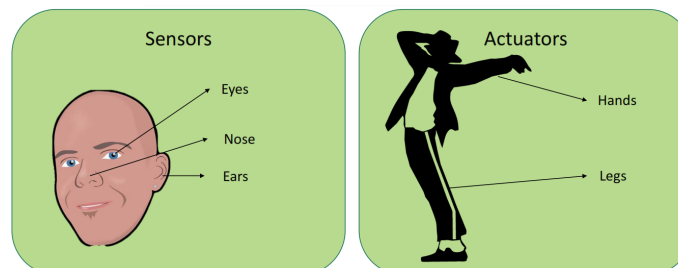
- Robot
- Pabrik
- Program *Web Shopping*

Computational agents yang berperilaku secara otonom.



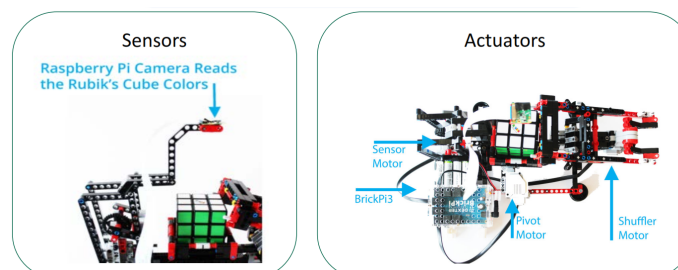
Gambar 2.1: Agent dan Environment

Sebagai contoh pada manusia:



Gambar 2.2: Human Agent

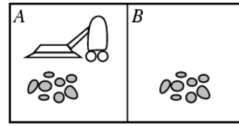
Adapun contoh pada robot:



Gambar 2.3: Rubik Solver Robot Agent

2.2 Agent Model (Studi Kasus)

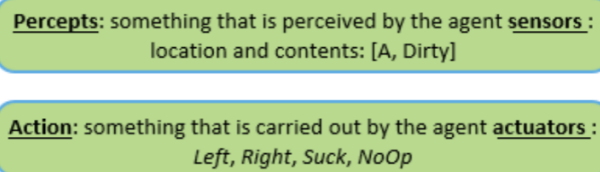
Berikut persoalan yang sederhana, yaitu Agent Penghisap Debu. Terdapat 2 buah ruangan, A dan B. Agent pada awalnya berada pada ruangan A.



Gambar 2.4: Vacuum-Cleaner World

Dari persoalan, maka dapat diperoleh:

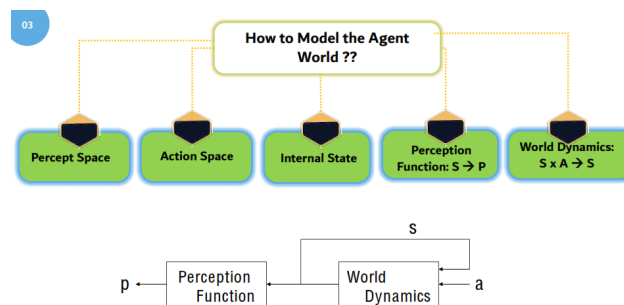
1. Agent dapat menangkap dua hal dari lingkungannya, yaitu lokasi dan keadaan lingkungannya.
2. Aksi yang dapat dilakukan oleh agen adalah bergerak, menghisap, dan *do nothing*



Gambar 2.5: Persepsi dan Aksi Agent

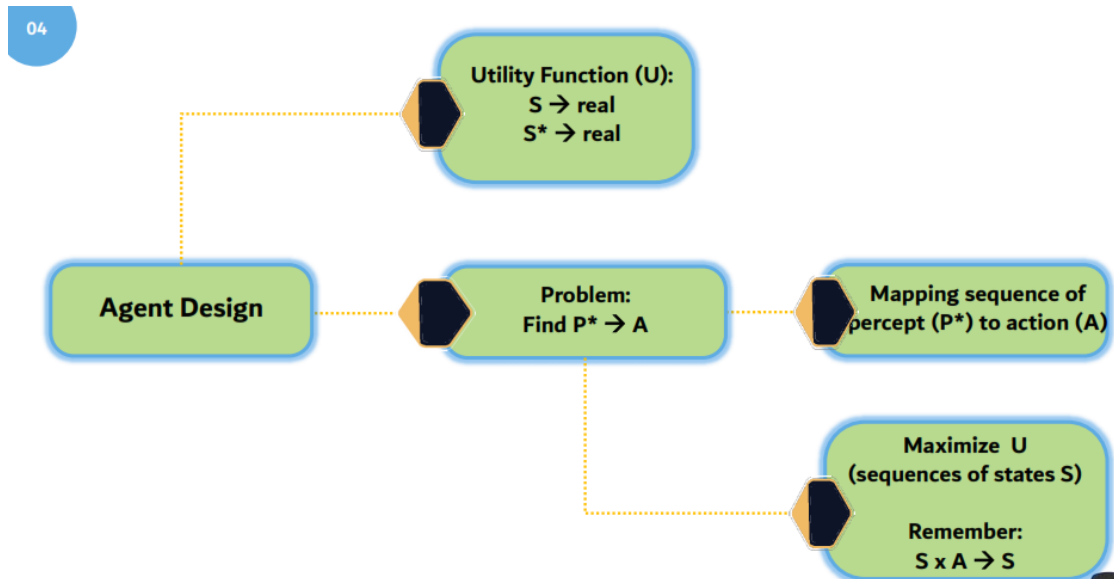
Dari analisis di atas:

1. Agen memiliki kumpulan persepsi atau kondisi atau state yang bisa ditangkap oleh agen yang dirancang ada pada agen (Percept Space)
2. Aksi apa aja yang bisa dilakukan oleh agen untuk mencapai tujuannya juga harus dirancang dari awal (Action Space)
3. Kondisi Internal atau state dalam program atau proses berpikir agen untuk menentukan apakah goal telah tercapai atau belum (Internal Space)
4. Representasi state mungkin berbeda dari kondisi yang ditangkap oleh agen, maka diperlukan fungsi yang memetakan state ke dalam suatu kondisi yang bisa ditangkap oleh agen (Perception Function: $S \rightarrow P$)
5. Dinamika dunia agen, jika pada suatu state lingkungan dilakukan aksi tertentu, maka state lingkungan akan berubah atau tetap (World Dynamics: $S \times A \rightarrow S$)



Gambar 2.6: Pemodelan Dunia Agent

Setelah itu, perlu suatu angka kualitas yang menunjukkan seberapa baik agen yang dirancang dengan menggunakan nilai utilitas.



Gambar 2.7: Evaluasi Agent

2.3 Rational Agent

Agen perlu untuk bekerja dengan baik dengan selalu berpikir untuk memilih aksi terbaik dalam mencapai tujuan yang diinginkan. Ketika agen memilih aksi terbaik, perlu didefinisikan *performance measure*.

Definisi 2.2: Rational Agent

Untuk tiap *sequence percepts* yang mungkin dan modal/bekal pengetahuan yang dimiliki oleh agen, **Rational Agent** harus memilih aksi yang tepat yang dapat memaksimalkan *performance measure*

Notes

Rational tidak berarti maha tahu (omscience).

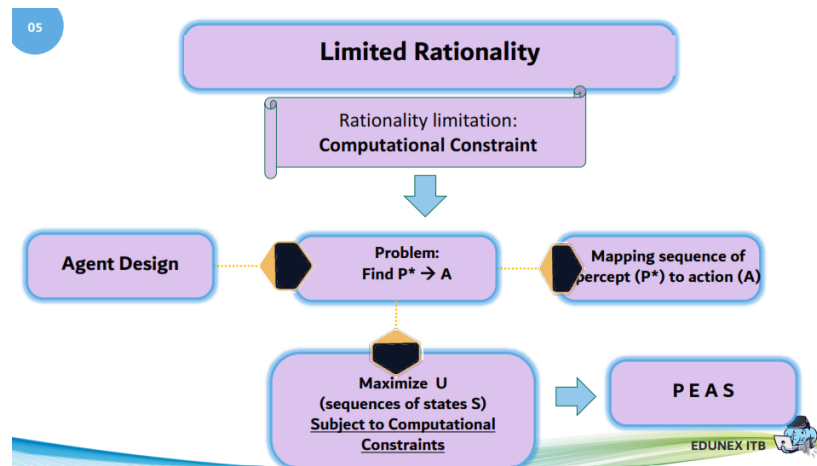
Jika seseorang tidak mendengarkan ramalan cuaca yang menyatakan hari ini akan curah dengan tidak ada hujan. Namun, seseorang tersebut masih membawa payung. Aksi tersebut masih dikatakan rational karena seseorang tidak tahu mengenai ramalan cuaca.

Rational juga tidak sama dengan sukses.

Ketika seseorang tersebut membawa payung agar tidak basah, tetapi seseorang tersebut bertemu dengan anjing di tengah jalan dan menggunakan payung tersebut untuk menghalau anjing.

Seseorang tersebut sukses menggunakan payung untuk menghindari digigit anjing, tetapi tidak rasional karena payung tersebut untuk menghalau hujan, bukan untuk menghalau anjing.

Akibat batasan komputasi, maka definisi desain agen akan berubah pada pemilihan aksi untuk mendapatkan *sequence of states* yang memiliki nilai utilitas maksimal.



Gambar 2.8: Definisi dengan PEAS

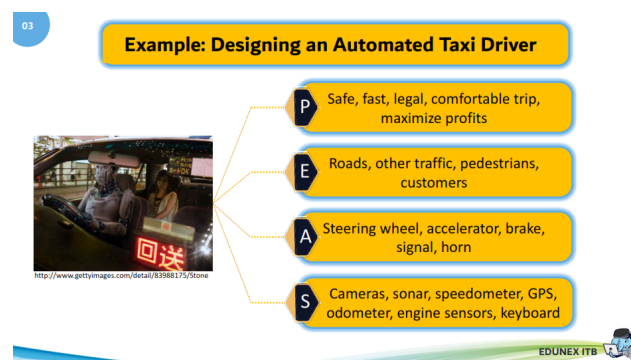
2.4 PEAS

Konsep

PEAS adalah kerangka kerja yang digunakan untuk mendefinisikan lingkungan tugas dalam AI, terdiri dari empat komponen:

1. Performance Measure - Kriteria untuk mengevaluasi keberhasilan agent, seperti akurasi.
2. Environment - Kondisi atau konteks di mana agent beroperasi, seperti dunia nyata atau simulasi.
3. Actuators - Mekanisme yang digunakan agent untuk bertindak, seperti motor atau output teks.
4. Sensors - Alat yang digunakan agent untuk memahami lingkungan, seperti kamera.

PEAS membantu merancang agent dengan memahami kebutuhan dan batasan lingkungan tugas secara terstruktur.

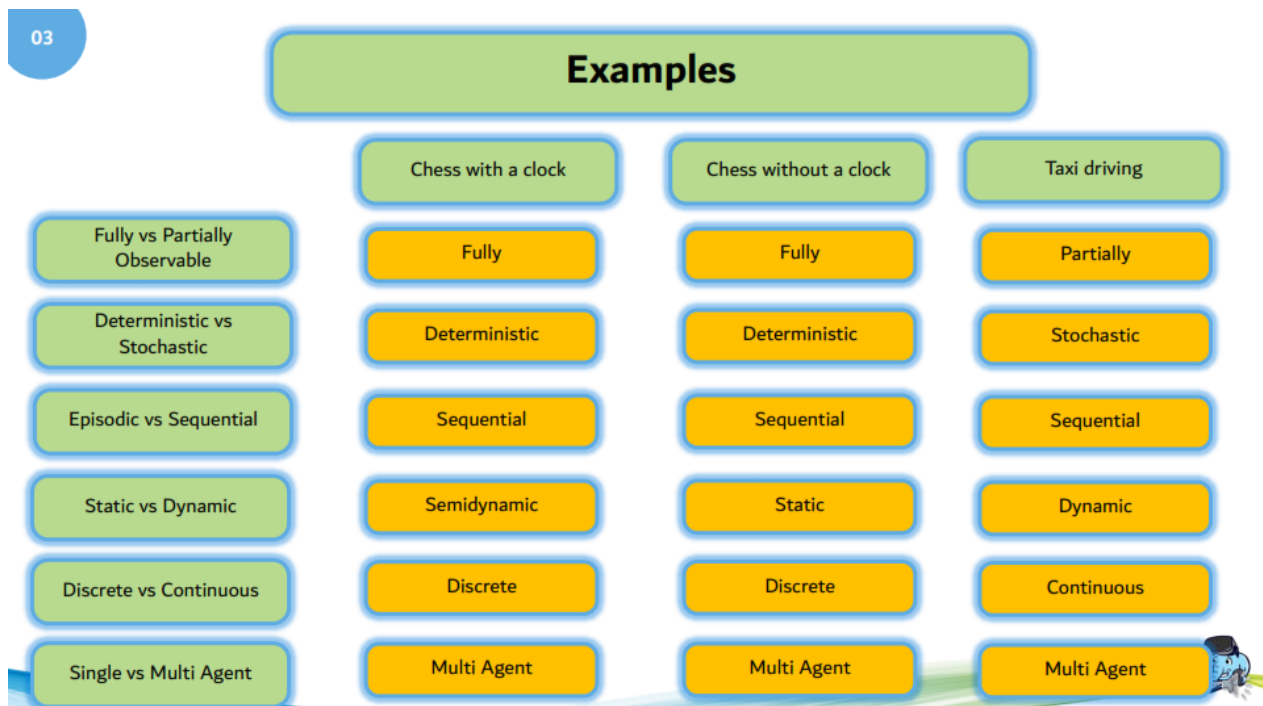


Gambar 2.9: Contoh Abstraksi PEAS

2.5 Task Environment

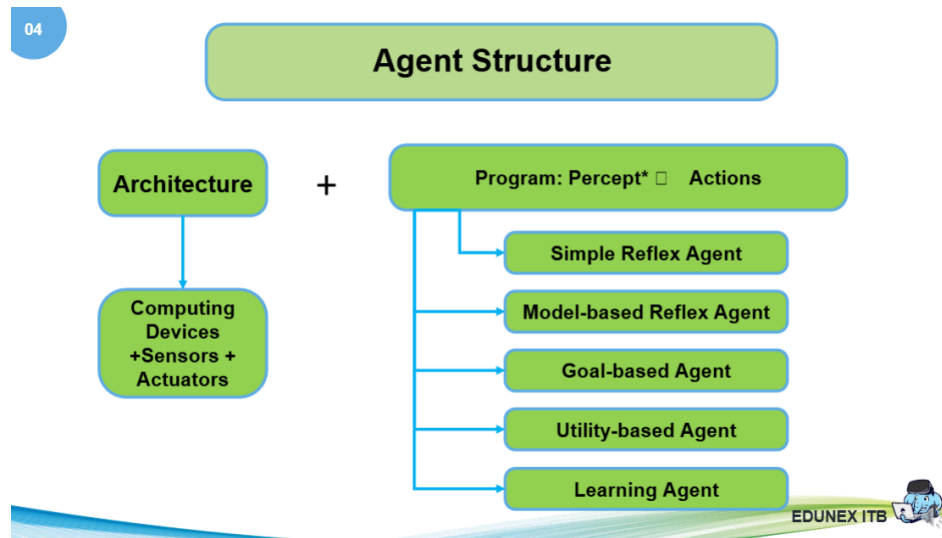
Enam properti lingkungan tugas dalam AI meliputi:

1. Fully Observable vs Partially Observable: Apakah agent memiliki akses penuh ke informasi lingkungan (fully) atau hanya sebagian (partially).
 2. Deterministic vs Stochastic: Apakah hasil tindakan dapat diprediksi (deterministic) atau mengandung elemen acak (stochastic).
 3. Episodic vs Sequential - Apakah tugas terdiri dari episode independen (episodic) atau langkah-langkah yang saling bergantung (sequential).
 4. Static vs Dynamic vs Semidynamic: Apakah lingkungan tetap (static) atau berubah seiring waktu (dynamic) atau lingkungan tetap tetapi performanya dinilai berdasarkan waktu yang digunakan (semidynamic) seperti catur dengan batasan waktu.
 5. Discrete vs Continuous: Apakah state, aksi, dan waktu bersifat diskrit (discrete) atau kontinu (continuous).
 6. Single Agent vs Multi-Agent: Apakah hanya ada satu agent atau beberapa agent yang berinteraksi.
- X. Known vs Unknown: Lebih pada pengetahuan agen terhadap cara kerja lingkungan, keluaran diberikan (known) atau harus dipelajari agen (unknown)



Gambar 2.10: Studi Kasus Task Environment

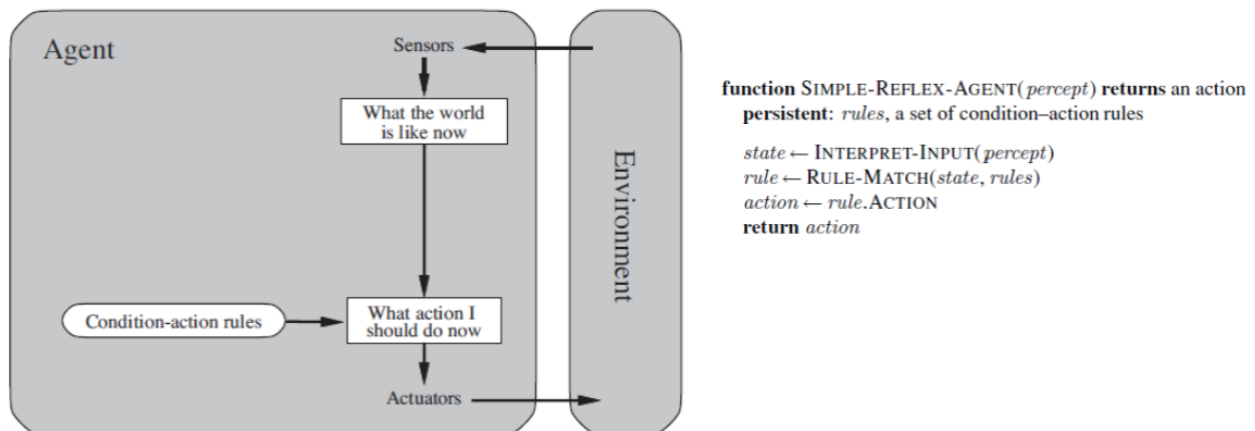
2.6 Agent Structure & Types



Gambar 2.11: Arsitektur dan Tipe-tipe Agent

Konsep: Simple Reflex Agent

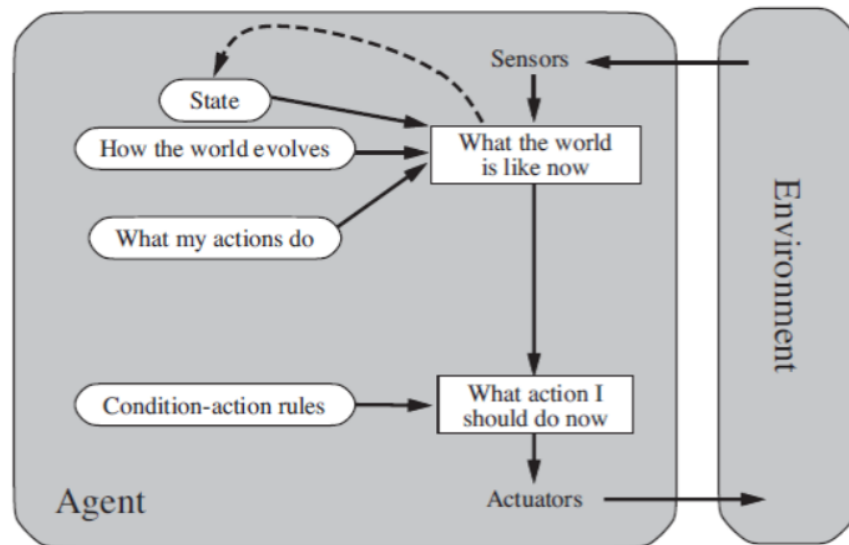
Agent yang bertindak berdasarkan aturan kondisi-aksi sederhana, hanya menggunakan persepsi saat ini (contoh: termostat).



Gambar 2.12: Simple Reflex Agent

Konsep: Model-Based Reflex Agent

Agent yang menggunakan model internal lingkungan untuk melacak state berdasarkan persepsi masa lalu dan saat ini (contoh: vacuum cleaner cerdas).



function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
persistent: *state*, the agent's current conception of the world state
model, a description of how the next state depends on current state and action
rules, a set of condition–action rules
action, the most recent action, initially none

```

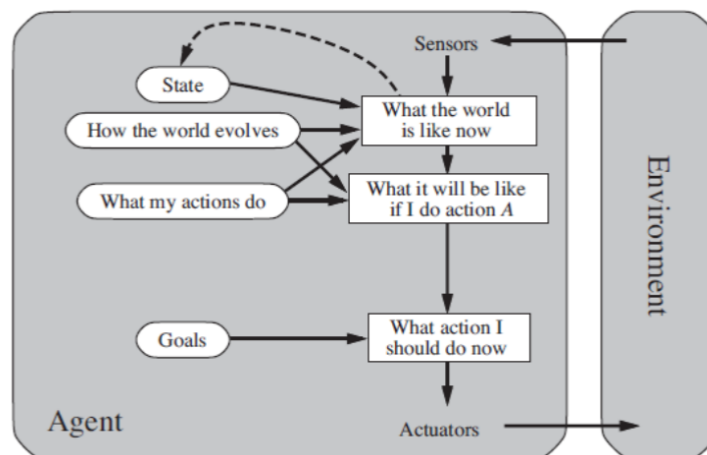
state ← UPDATE-STATE(state, action, percept, model)
rule ← RULE-MATCH(state, rules)
action ← rule.ACTION
return action

```

Gambar 2.13: Model-Based Reflex Agent

Konsep: Goal-Based Agent

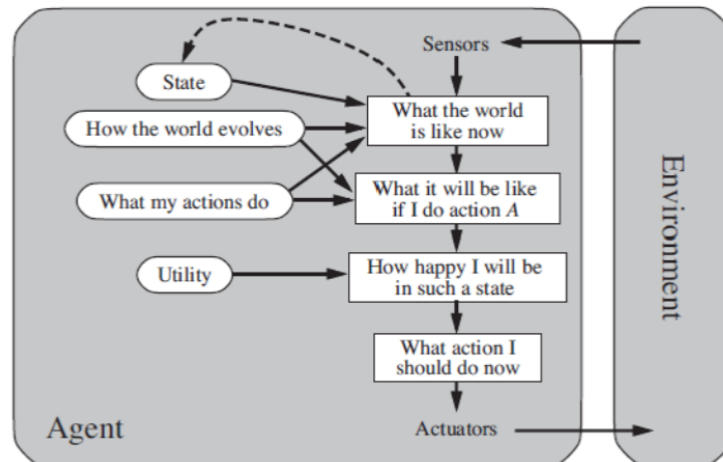
Agent yang bertindak untuk mencapai tujuan tertentu dengan mempertimbangkan konsekuensi aksi (contoh: sistem navigasi GPS).



Gambar 2.14: Goal-Based Agent

Konsep: Utility-Based Agent

Agent yang memaksimalkan utilitas atau kepuasan berdasarkan fungsi utilitas, mempertimbangkan preferensi (contoh: agen e-commerce yang merekomendasikan produk).



Gambar 2.15: Utility-Based Agent

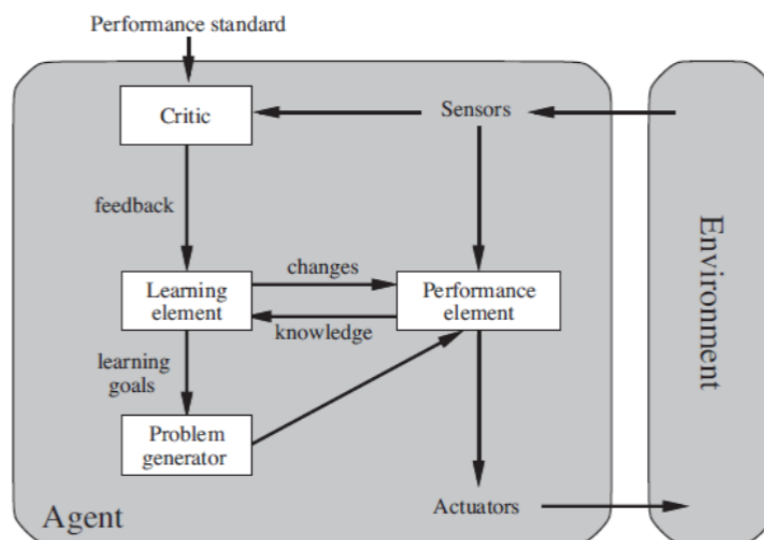
Konsep: Learning Agent

Agent yang dapat belajar dari pengalaman untuk meningkatkan performa seiring waktu, menggunakan komponen seperti kritik dan learning (contoh: agen reinforcement learning untuk permainan).

Critic tells the learning element how well the agent is doing with respect to a fixed performance standard

The **learning element** (responsible for making improvement) uses feedback from the critic on how the agent is doing and determines how the performance element should be modified to do better in the future

Problem Generator is responsible for suggesting actions that will lead to new and informative experiences

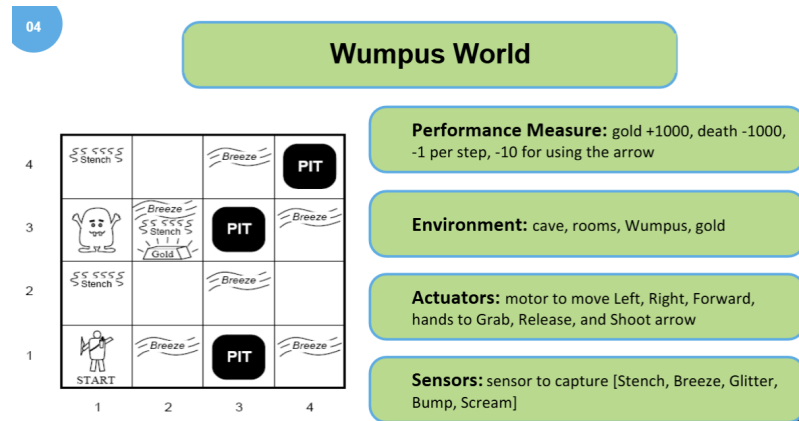


Performance element responsible for element selecting external actions. The performance element is what we have previously considered to be the entire agent: it takes in percepts and decides on actions

Gambar 2.16: Learning Agent

2.7 Agent Level (Studi Kasus)

Untuk memperkenalkan tingkatan pada agent, terdapat persoalan klasik, yaitu Wumpus World. Terdapat goa yang memiliki banyak ruangan, Wumpus akan memakan apa aja dan siapa aja yang masuk ke ruangnya. Wumpus dapat dipanah oleh agen, tetapi agen hanya memiliki satu anak panah. Pada beberapa ruangan goa, terdapat lubang tidak berdasar yang akan menjebak siapa aja kecuali Wumpus. Tujuan dari agent adalah mendapatkan emas yang ada pada salah satu ruangan goa.



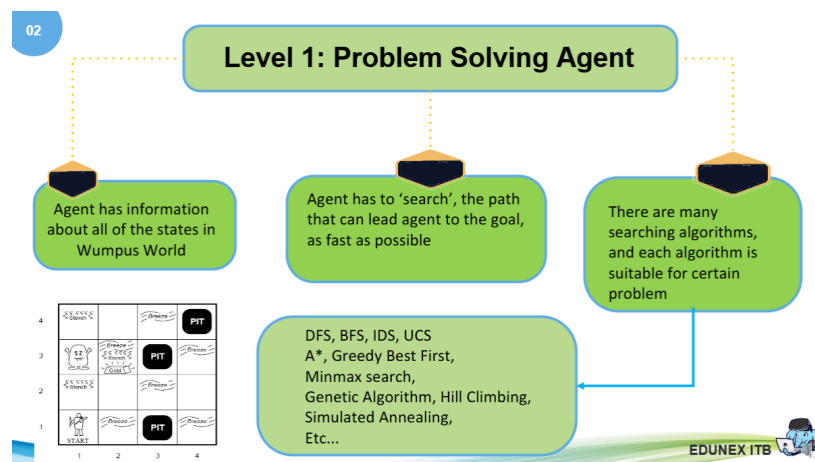
Gambar 2.17: Persoalan Wumpus World dan Abstraksi PEAS-nya

Konsep: Problem Solving Agent (Level 1)

Semua state persoalan yang bisa ditangkap oleh agent dan semua aksi yang bisa dipilih untuk mencapai tujuan telah diberikan.

Tugas agent adalah melakukan pencarian aksi apa yang harus dilakukan agar agent dapat mendapatkan goal dari start.

Agent paling sederhana dalam pemrosesan pencarian solusi dan tidak ada penalaran khusus.

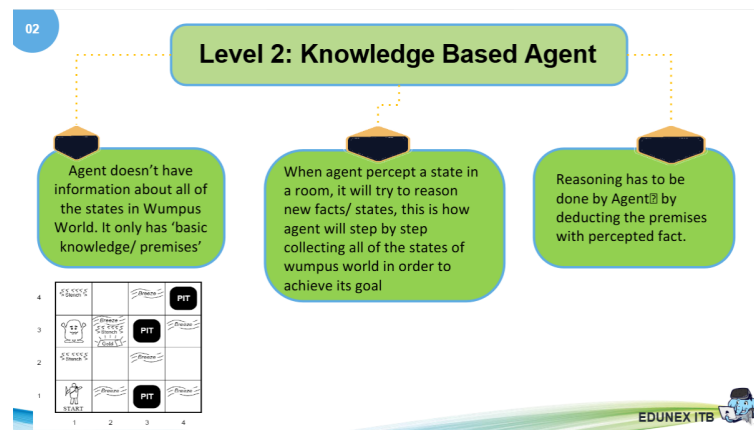


Gambar 2.18: Problem Solving Agent pada persoalan Wumpus World

Konsep: Knowledge Based Agent (Level 2)

Tidak semua state diberikan di awal, namun yang diberikan adalah pengetahuan dasar atau premis yang bisa dimanfaatkan pada proses penalaran untuk mendapatkan fakta atau state yang baru.

Agent telah dilengkapi oleh pengetahuan. Oleh karena itu, agent memiliki kemampuan penalaran untuk memperoleh fakta atau state baru dari dunia persoalan yang dihadapi dalam rangka untuk menyelesaikan persoalan tersebut.

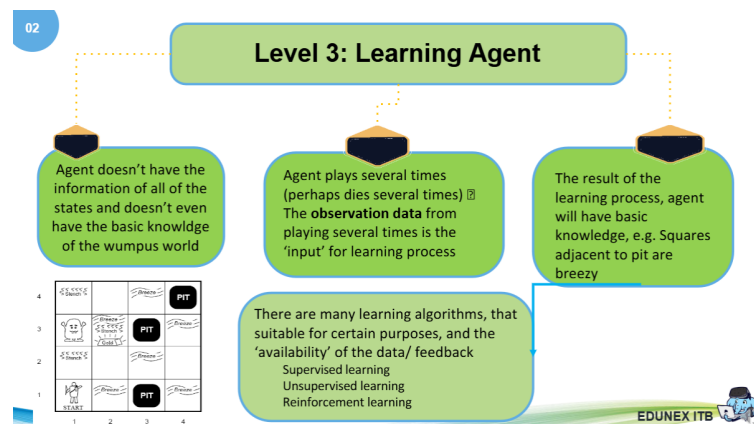


Gambar 2.19: Knowledge Based Agent pada persoalan Wumpus World

Konsep: Learning Agent (Level 3)

Tidak ada informasi yang diberikan kepada agent mengenai states yang ada di persoalan. Modal yang diberikan kepada agent adalah kumpulan data hasil observasi selama beberapa waktu.

Dari kumpulan data tersebut, agent harus dapat membentuk pengetahuan dasar dan menyelesaikan persoalan.



Gambar 2.20: Learning Agent pada persoalan Wumpus World

Bab 3

Local Search

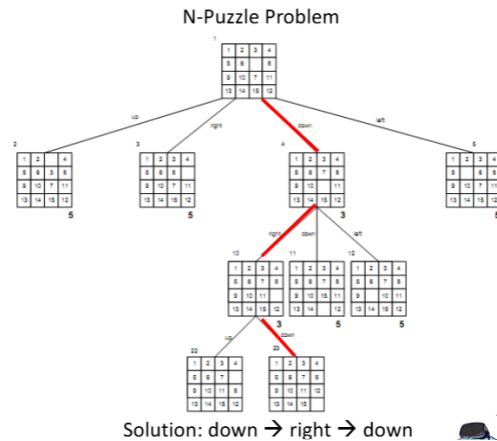
3.1	Classical Search Review	24
3.2	Local Search	24
3.3	Landscape Ruang State pada Local Search	26
3.4	State	26
3.5	Successor dan Neighbor	27
3.6	Hill Climb	27
3.7	Simulated Annealing	30
3.8	Genetic Algorithm	31

Local Search merupakan metode pencarian generik yang berusaha mengoptimisasi pencarian berdasarkan *local changes* saja, bukan space solusi secara keseluruhan. Ditulis oleh Muhammad Fathur Rizky saat liburan.

3.1 Classical Search Review

Definisi 3.1

Dirancang untuk lingkungan yang **observable** dan **deterministic** dengan melakukan eksplorasi pencarian secara sistematis sehingga dihasilkan solusinya berupa path atau urutan aksi.



Gambar 3.1: Classical Search pada persoalan N-puzzle

Akan tetapi, pada beberapa persoalan solusi berupa *path to goal* tidak relevan. Sebagai contoh, persoalan N-queen yang solusinya berupa konfigurasi akhir posisi masing-masing *queen*.



Gambar 3.2: Persoalan N-queen

3.2 Local Search

Definisi 3.2

Pencarian Lokal adalah kelas algoritma di mana jalur menuju tujuan tidak penting. Algoritma ini beroperasi pada formulasi keadaan lengkap, artinya setiap keadaan merepresentasikan solusi yang utuh.

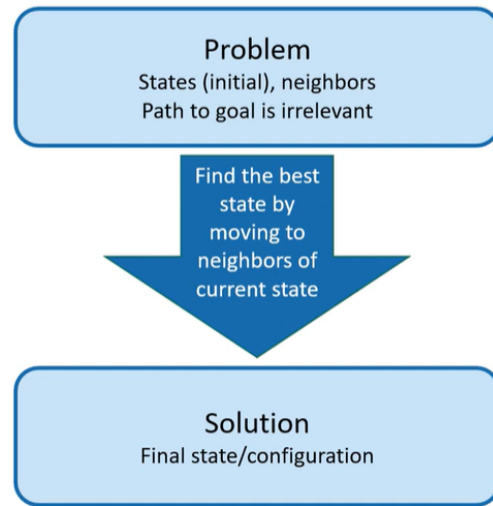
Pencarian dimulai dari satu keadaan saat ini dan hanya bergerak ke tetangga dari keadaan tersebut. Solusinya adalah konfigurasi keadaan akhir, bukan urutan tindakan. Nilai keadaan ditentukan oleh fungsi objektif atau fungsi biaya heuristik.

Tujuannya adalah untuk menemukan keadaan terbaik, atau optimum global, dalam lanskap ruang keadaan.

04

Local Search

- If path to goal does not matter, we might consider different class of algorithms.
- Local search: complete state formulation
 - State: "complete" configurations
 - Keep single current state, not paths.
 - Action: move only to neighbors of current state.
 - No path cost → state value: value according to objective function or heuristic cost function
 - No goal test → maximum state value



EDUNEX ITB



Gambar 3.3: Local Search pada N-queen

Terus bagaimana perbedaan antara kedua pencarian tersebut pada persoalan N-queen?



Gambar 3.4: Perbedaan kedua pencarian pada persoalan N-queen

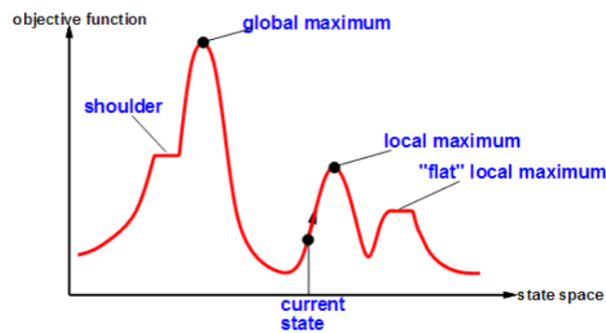
3.3 Landscape Ruang State pada Local Search

Konsep

Sebuah landscape memiliki “location” (didefinisikan oleh state) dan “elevation” (nilai dari **objective** atau **heuristic cost value**).

Local search bertujuan untuk menemukan **global optimum**

Masalah: tergantung pada initial state, dapat **terjebak** dalam **local optimum**.

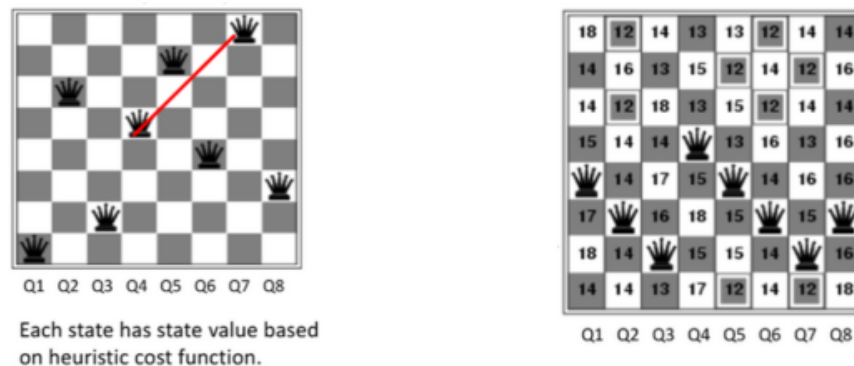


Gambar 3.5: Grafik Local Search

3.4 State

Konsep

Berupa konfigurasi lengkap atau semua variabelnya telah diberi nilai.



Gambar 3.6: Contoh sebuah *state* pada persoalan N-queen

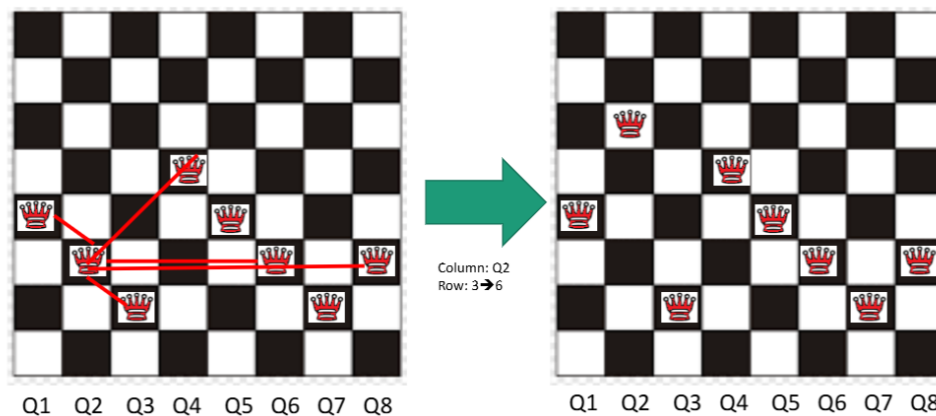
3.5 Successor dan Neighbor

Konsep

Successor function akan menghasilkan seluruh state yang mungkin dari melakukan hanya satu aksi (**Successor**).

Neighbor simpelnya adalah **successor** yang akhirnya dipilih, bisa dengan random ataupun dipilih berdasarkan nilai heuristiknya yang tertinggi.

Neighbor: Random Successor



Gambar 3.7: Neighbor jika dari random successor pada persoalan N-queen

3.6 Hill Climb

Konsep

Hill-Climbing adalah algoritma pencarian lokal yang secara terus-menerus bergerak ke arah peningkatan nilai (untuk fungsi objektif) atau penurunan nilai (untuk biaya). Pencarian dimulai dari keadaan awal yang dibuat secara acak dan berhenti ketika mencapai "puncak" di mana tidak ada tetangga yang memiliki nilai lebih tinggi. Algoritma ini bisa terjebak di maksimum lokal, tergantung pada keadaan awalnya.

3.6.1 Basic Hill Climb

Konsep

Varian ini juga dikenal sebagai *steepest-ascent*. Algoritma ini akan memilih tetangga dengan nilai terbaik dari keadaan saat ini. Jika nilai tetangga terbaik tidak lebih baik dari nilai saat ini, algoritma akan berhenti.

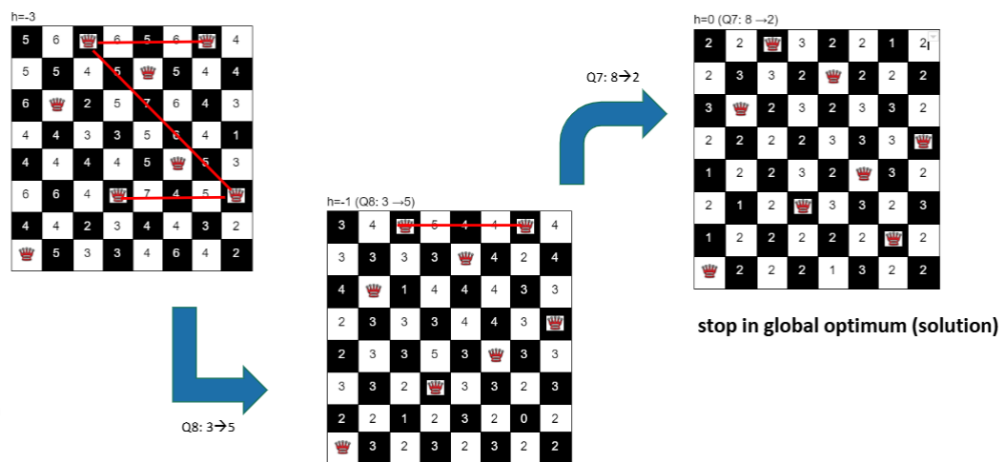
Algorithm 1 Basic Hill-Climbing (Steepest-Ascent)

```

1: function BASICHILLCLIMB(keadaan_awal)
2:   keadaan_sekarang  $\leftarrow$  keadaan_awal
3:   loop
4:     tetangga  $\leftarrow$  tetangga dengan nilai tertinggi dari keadaan_sekarang
5:     if nilai(tetangga)  $\leq$  nilai(keadaan_sekarang) then
6:       return keadaan_sekarang
7:     end if
8:     keadaan_sekarang  $\leftarrow$  tetangga
9:   end loop
10: end function

```

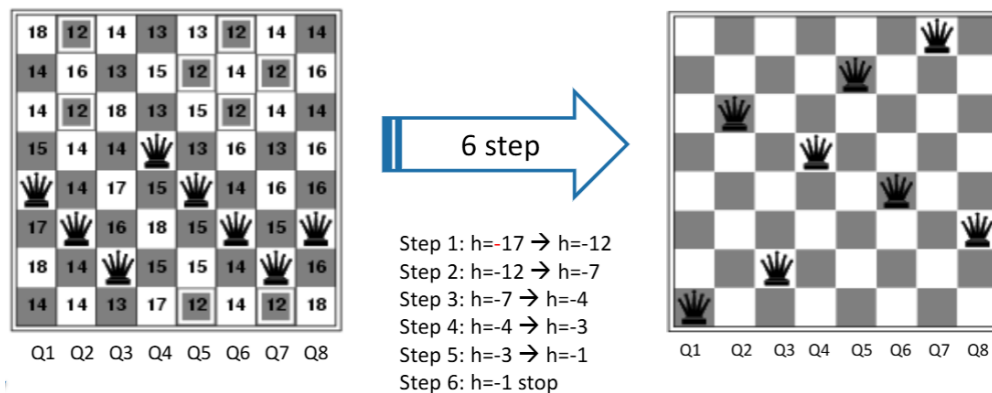
Hill-climbing: Illustration



Gambar 3.8: Ilustrasi *steepest-ascent* Hill Climb Search yang berhasil menuju *global optimum*

Akan tetapi, tidak semua kasus akan berujung dengan *ending* baik.

Hill-climbing: Stuck In Local Optimum



Gambar 3.9: Ilustrasi *steepest-ascent* Hill Climb Search yang nge-stuck

Oleh karena itu, terdapat varian-varian dari Hill Climb Search dengan tujuan menghindari *stuck* pada *local optimum*.

Hill-climbing for 8-Queen Problem

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

State space:

$8^8 \approx 16.8$ million states

Average case: works quickly
when **success**: avg 4 steps
when **stuck**: avg 3 steps

Best case:
initial state = goal state
Prob: $92 / 8^8 = 0.00054\%$

Get **stuck** 86%,
solving only 14% of
problem instances

Gambar 3.10: Evaluasi Basic Hill Climb Search

3.6.2 Sideways

Konsep

Varian ini mengizinkan gerakan menyamping, yaitu pindah ke tetangga yang memiliki nilai yang sama dengan keadaan saat ini, dengan batasan jumlah langkah untuk menghindari perulangan.

Algorithm 2 Hill-Climbing with Sideways Moves

```

1: function HILL_CLIMB_SIDEWAYS(keadaan_awal, batas_menyamping)
2:   keadaan_sekarang  $\leftarrow$  keadaan_awal
3:   jumlah_menyamping  $\leftarrow$  0
4:   loop
5:     tetangga  $\leftarrow$  tetangga dengan nilai tertinggi dari keadaan_sekarang
6:     if nilai(tetangga) < nilai(keadaan_sekarang) then
7:       keadaan_sekarang  $\leftarrow$  tetangga
8:       jumlah_menyamping  $\leftarrow$  0
9:     else if nilai(tetangga) = nilai(keadaan_sekarang) and jumlah_menyamping <
       batas_menyamping then
10:      keadaan_sekarang  $\leftarrow$  tetangga
11:      jumlah_menyamping  $\leftarrow$  jumlah_menyamping + 1
12:     else
13:       return keadaan_sekarang
14:     end if
15:   end loop
16: end function

```

3.6.3 Stochastic

Konsep

Pada **Stochastic Hill-Climbing**, algoritma memilih satu penerus secara acak dan hanya pindah jika nilainya lebih baik.

Algorithm 3 Stochastic Hill-Climbing

```

1: function STOCHASTIC_HILL_CLIMB(keadaan_awal, langkah_maks)
2:   keadaan_sekarang  $\leftarrow$  keadaan_awal
3:   for  $i \leftarrow 1$  to langkah_maks do
4:     tetangga_acak  $\leftarrow$  pilih tetangga acak dari keadaan_sekarang
5:     if nilai(tetangga_acak) > nilai(keadaan_sekarang) then
6:       keadaan_sekarang  $\leftarrow$  tetangga_acak
7:     end if
8:   end for
9:   return keadaan_sekarang
10: end function

```

3.6.4 Random Restart

Konsep

Strategi ini menjalankan serangkaian pencarian hill-climbing dari keadaan awal acak yang berbeda.

Algorithm 4 Random-Restart Hill-Climbing

```

1: function RANDOM_RESTART_HILL_CLIMB(masalah, hill_climb_func, restart_maks)
2:   for  $i \leftarrow 1$  to restart_maks do
3:     keadaan_awal  $\leftarrow$  masalah.buat_keadaan_acak()
4:     solusi  $\leftarrow$  hill_climb_func(keadaan_awal)
5:     if solusi adalah keadaan tujuan then
6:       return solusi
7:     end if
8:   end for
9:   return "Gagal menemukan solusi"
10: end function

```

3.7 Simulated Annealing

Konsep

Simulated Annealing adalah varian dari stochastic hill-climbing yang mengizinkan beberapa gerakan "menurun" untuk menghindari terjebak di maksimum lokal.

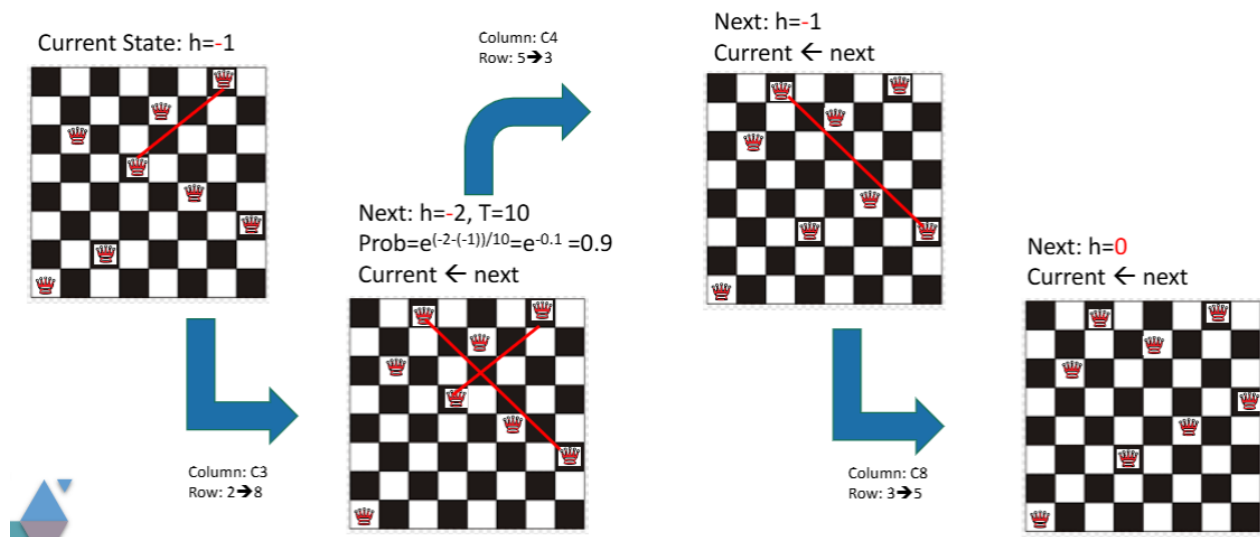
Algorithm 5 Simulated Annealing

```

1: function SIMULATED_ANNEALING(masalah, jadwal)
2:   keadaan_sekarang  $\leftarrow$  masalah.keadaan_awal
3:   for  $t \leftarrow 1$  to  $\infty$  do
4:      $T \leftarrow \text{jadwal}(t)$ 
5:     if  $T = 0$  then
6:       return keadaan_sekarang
7:     end if
8:     keadaan_selanjutnya  $\leftarrow$  pilih penerus acak dari keadaan_sekarang
9:      $\Delta E \leftarrow \text{nilai}(\text{keadaan_selanjutnya}) - \text{nilai}(\text{keadaan_sekarang})$ 
10:    if  $\Delta E > 0$  then
11:      keadaan_sekarang  $\leftarrow$  keadaan_selanjutnya
12:    else
13:      keadaan_sekarang  $\leftarrow$  keadaan_selanjutnya dengan probabilitas  $e^{\Delta E/T}$ 
14:    end if
15:  end for
16: end function

```

Simulated Annealing: Illustration



Gambar 3.11: Ilustrasi Simulated Annealing pada persoalan N-queens

3.8 Genetic Algorithm

Konsep

Algoritma Genetika (GA) adalah metode pencarian yang terinspirasi dari evolusi biologis. Algoritma ini memelihara populasi individu dan menggunakan operator seperti seleksi, pindah silang, dan mutasi untuk menghasilkan generasi baru.

Algorithm 6 Genetic Algorithm

```

1: function GENETIC_ALGORITHM(populasi, fungsi_kebugaran)
2:   repeat
3:     populasi_baru  $\leftarrow$  himpunan kosong
4:     for  $i \leftarrow 1$  to ukuran(populasi) do
5:        $x \leftarrow$  Pilih_Acak(populasi, fungsi_kebugaran)
6:        $y \leftarrow$  Pilih_Acak(populasi, fungsi_kebugaran)
7:       anak  $\leftarrow$  Reproduksi( $x, y$ )
8:       if (probabilitas acak kecil) then
9:         anak  $\leftarrow$  Mutasi(anak)
10:      end if
11:      tambahkan anak ke populasi_baru
12:    end for
13:    populasi  $\leftarrow$  populasi_baru
14:  until kondisi berhenti terpenuhi
15:  return individu terbaik dalam populasi
16: end function

```

Genetic Algorithm: Illustration (Russel & Norvig, 2010)

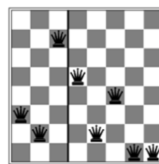
**Gambar 3.12:** Ilustrasi Genetic Algorithm pada persoalan N-queens**3.8.1 Individu****Definisi 3.3**

Sebuah **state** / **individu** direpresentasikan sebagai string pada finite alphabet (seringkali berupa string dari 0s dan 1s). Satu karakter - satu variabel.

State Value: Fitness Function

- Evaluation function (**fitness function**).
- Higher values for better states.
- Fitness function for n-queen: number of **non-attacking pairs** of queens
 - min = 0
 - max = $(8 \times 7)/2 = 28$ (**global maximum**)

32752411



F(32752411)=28-5=23

Gambar 3.13: Fitness Function untuk State Value

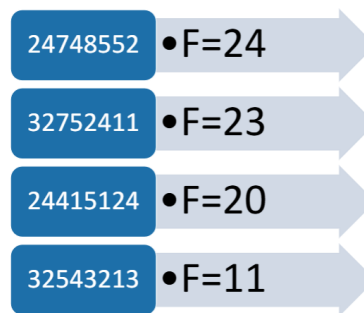
3.8.2 Populasi

Konsep

GA dimulai dengan k randomly generated states.

Sebuah successor state dihasilkan dengan menggabungkan dua parent states.

Menghasilkan next generation dari states melalui selection, crossover, dan mutation.

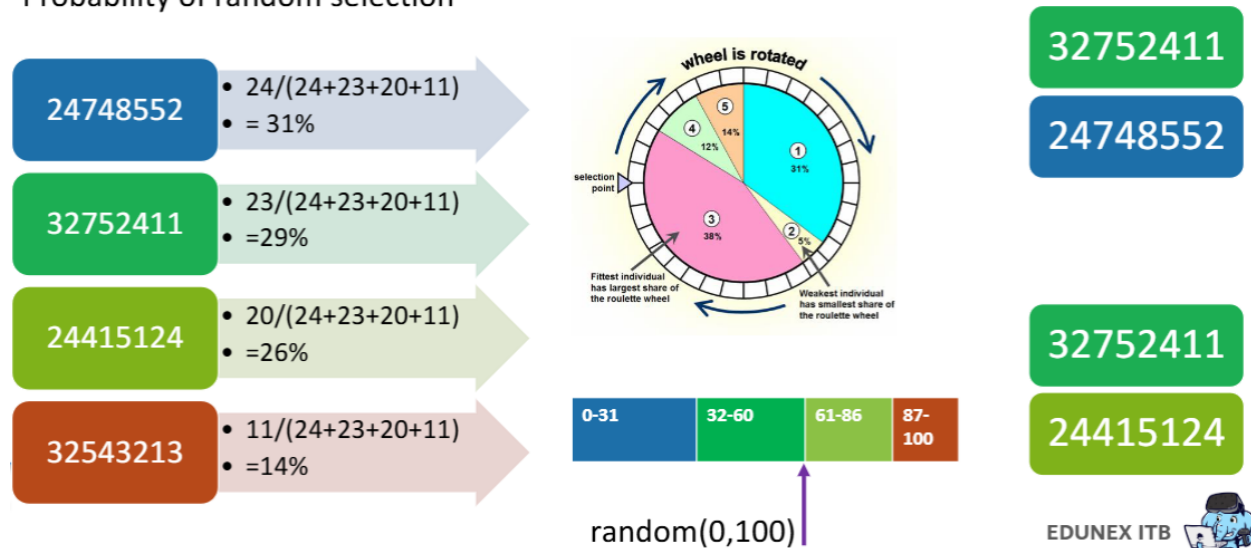


Gambar 3.14: Populasi dengan $k = 4$ individu

3.8.3 Selection

Random Selection of Parent States

Probability of random selection



Gambar 3.15: Proses Selection dengan random pada Genetic Algorithm

3.8.4 Cross Over & Mutation

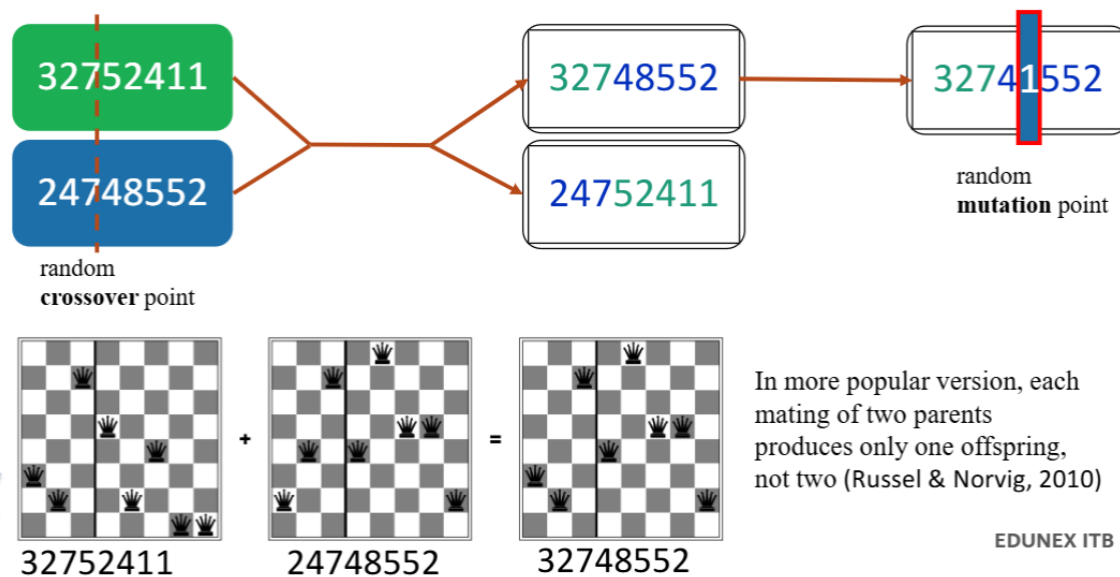
Konsep

Diambil *index/point* random sebagai titik pisah antar dua individu.

Tukar hasil pemisahan antar dua individu

Kemudian, untuk mutasi diambil *index/point* random sebagai posisi yang akan diganti dengan nilai random sesuai domain.

Cross Over / Reproduce & Mutation



Gambar 3.16: Proses Cross-over dan Mutation pada Genetic Algorithm

Bab 4

Adversarial Search

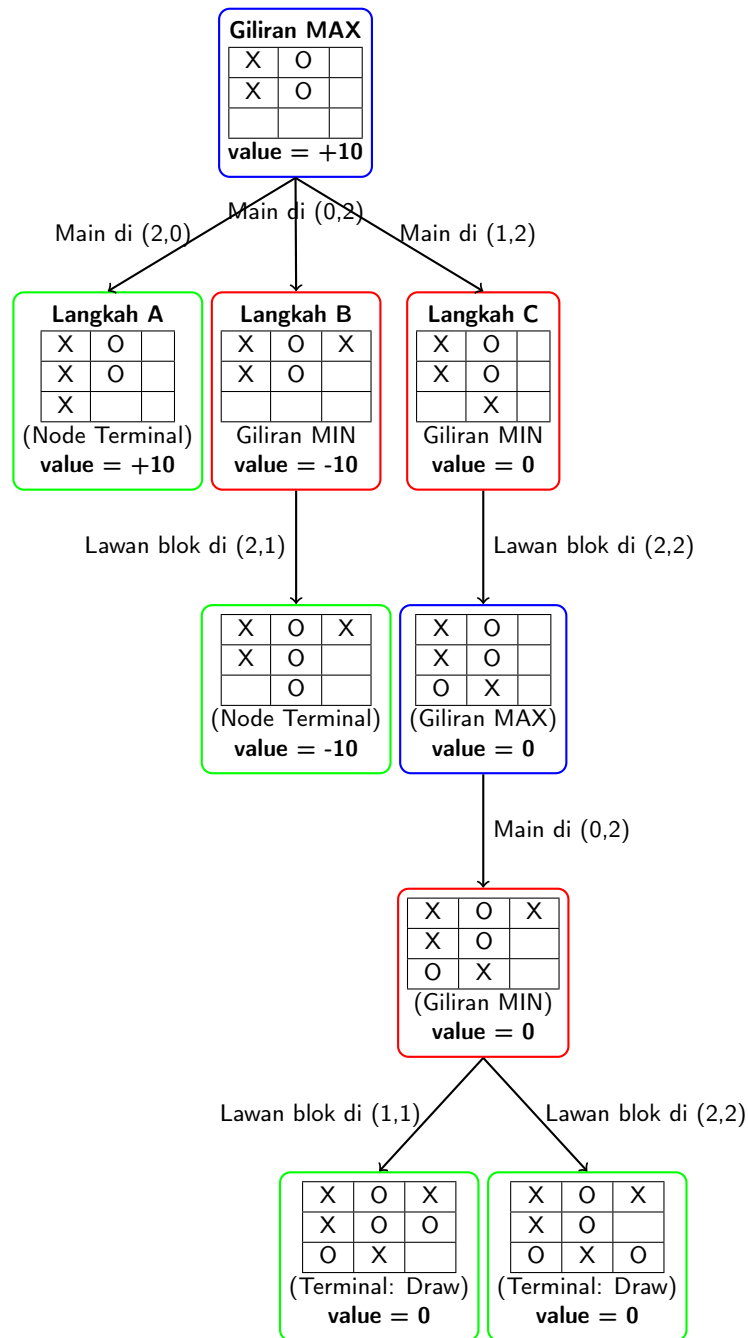
4.1	Minimax Algorithm	36
4.2	Alpha-Beta Pruning	38

Algoritma pencarian adversarial merupakan tulang punggung pengambilan keputusan strategis dalam kecerdasan buatan. Algoritma ini memungkinkan agen untuk menavigasi skenario kompetitif secara efektif.

Adversarial search sangat cocok digunakan pada kondisi di mana terdapat dua atau lebih agen yang berkompetisi. Dengan kata lain, ini biasanya diterapkan pada permainan strategi. Dengan menggunakan adversarial search, agen dapat membuat sebuah keputusan sambil mengantisipasi countermove musuh.

Ditulis oleh Muhammad Fathur Rizky.

4.1 Minimax Algorithm



Gambar 4.1: Diagram pohon keputusan Minimax. AI (MAX) akan memilih langkah A karena memiliki nilai tertinggi (+10).

Konsep

Algoritma Minimax adalah algoritma pengambilan keputusan yang digunakan dalam permainan dua pemain dengan informasi sempurna dan hasil zero-sum (kemenangan satu pemain adalah kekalahan pemain lain). Contoh klasiknya adalah Catur, Tic-Tac-Toe, dan Go. Tujuannya adalah untuk menemukan langkah optimal dengan mengasumsikan bahwa lawan juga akan bermain secara optimal.

$$\begin{cases} \text{evaluate}(n), & \text{jika } n \text{ adalah node terminal} \\ \max_{s \in \text{successors}(n)} \text{value}(s), & \text{jika } n \text{ adalah node MAX} \\ \min_{s \in \text{successors}(n)} \text{value}(s), & \text{jika } n \text{ adalah node MIN} \end{cases}$$

Algoritma ini bekerja dengan menetapkan dua peran: pemain Maximizer (MAX), yang tujuannya adalah memaksimalkan skornya, dan pemain Minimizer (MIN), yang diasumsikan akan selalu memilih langkah untuk meminimalkan skor MAX. Prosesnya dimulai dari keadaan akhir permainan (node terminal), di mana sebuah nilai numerik (disebut evaluate) diberikan, misalnya +1 untuk kemenangan MAX, -1 untuk kekalahan, dan 0 untuk seri.

Kemudian, secara rekursif mundur, pada setiap giliran MAX, algoritma akan memilih langkah yang mengarah ke keadaan dengan nilai maksimum. Sebaliknya, pada setiap giliran MIN, algoritma akan memilih langkah yang mengarah ke keadaan dengan nilai minimum. Dengan mensimulasikan pilihan optimal lawan, algoritma ini memungkinkan pemain untuk memilih langkah yang paling menguntungkan.

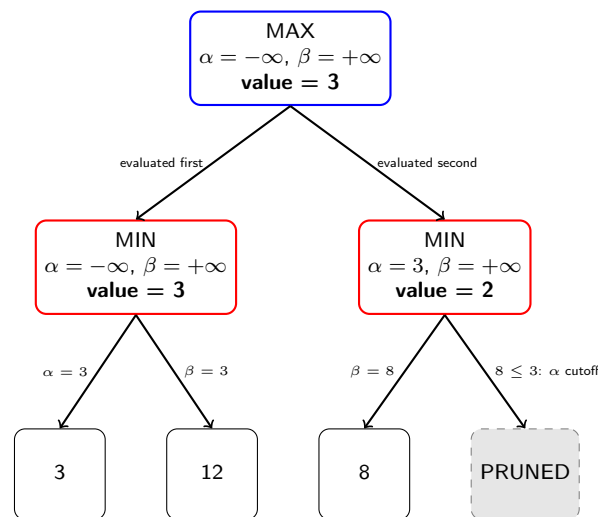
Dalam konteks contoh Tic-Tac-Toe pada Gamabr 4.1, Langkah A dipilih karena langsung menghasilkan kemenangan dengan nilai +10 tanpa memberikan kesempatan kepada lawan untuk merespons. Sementara itu, Langkah B dan Langkah C hanya menghasilkan nilai 0 (seri) karena pada giliran berikutnya, lawan (MIN) akan memilih langkah yang menghalangi kemenangan MAX, yaitu memblokir jalur yang dapat menyebabkan kekalahan.

Keoptimalan algoritma Minimax terjamin karena beberapa alasan fundamental. Pertama, algoritma ini melakukan eksplorasi menyeluruh terhadap semua kemungkinan langkah hingga mencapai keadaan terminal, sehingga tidak ada kemungkinan yang terlewat. Kedua, asumsi rasionalitas lawan memastikan bahwa evaluasi dilakukan terhadap skenario terburuk yang mungkin dihadapi, sehingga keputusan yang diambil akan tetap optimal meskipun lawan bermain dengan strategi terbaik mereka. Ketiga, sifat deterministik dari algoritma memastikan bahwa untuk kondisi permainan yang sama, hasil yang diperoleh akan selalu konsisten.

4.2 Alpha-Beta Pruning

Konsep: Definisi dan Konsep Dasar

Alpha-Beta Pruning merupakan teknik optimasi yang digunakan untuk meningkatkan efisiensi algoritma Minimax dengan mengeliminasi cabang-cabang pohon pencarian yang tidak perlu dievaluasi. Teknik ini mempertahankan hasil yang sama dengan algoritma Minimax standar namun dengan kompleksitas waktu yang jauh lebih rendah. Prinsip fundamental Alpha-Beta Pruning adalah menghentikan evaluasi suatu cabang ketika sudah dapat dipastikan bahwa cabang tersebut tidak akan mempengaruhi keputusan akhir.



Gambar 4.2: Ilustrasi proses Alpha-Beta Pruning. Node kedua dari kanan tidak perlu dievaluasi karena kondisi α cutoff terpenuhi.

4.2.1 Mekanisme Kerja

Alpha-Beta Pruning bekerja dengan memelihara dua nilai parameter selama proses pencarian:

Alpha (α): Merepresentasikan nilai terbaik yang dapat dijamin oleh pemain MAX pada jalur yang telah dievaluasi. Nilai ini dimulai dari negatif tak hingga dan akan meningkat seiring dengan ditemukannya langkah yang lebih baik bagi MAX.

Beta (β): Merepresentasikan nilai terbaik yang dapat dijamin oleh pemain MIN pada jalur yang telah dievaluasi. Nilai ini dimulai dari positif tak hingga dan akan menurun seiring dengan ditemukannya langkah yang lebih baik bagi MIN.

Proses pruning terjadi ketika kondisi $\alpha \geq \beta$ terpenuhi. Kondisi ini mengindikasikan bahwa cabang saat ini tidak akan dipilih karena sudah terdapat alternatif yang lebih baik di tempat lain dalam pohon pencarian.

Algorithm 7 Alpha-Beta Pruning

```

1: function ALPHA-BETA-SEARCH(state)
2:    $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
3:   return the action in  $\text{ACTIONS}(\text{state})$  with value  $v$ 
4: end function
5:
6: function MAX-VALUE(state,  $\alpha$ ,  $\beta$ )
7:   if  $\text{TERMINAL-TEST}(\text{state})$  then
8:     return  $\text{UTILITY}(\text{state})$ 
9:   end if
10:   $v \leftarrow -\infty$ 
11:  for each  $a$  in  $\text{ACTIONS}(\text{state})$  do
12:     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(\text{state}, a), \alpha, \beta))$ 
13:    if  $v \geq \beta$  then
14:      return  $v$   $\triangleright \beta$  cutoff
15:    end if
16:     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
17:  end for
18:  return  $v$ 
19: end function
20:
21: function MIN-VALUE(state,  $\alpha$ ,  $\beta$ )
22:   if  $\text{TERMINAL-TEST}(\text{state})$  then
23:     return  $\text{UTILITY}(\text{state})$ 
24:   end if
25:   $v \leftarrow +\infty$ 
26:  for each  $a$  in  $\text{ACTIONS}(\text{state})$  do
27:     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(\text{state}, a), \alpha, \beta))$ 
28:    if  $v \leq \alpha$  then
29:      return  $v$   $\triangleright \alpha$  cutoff
30:    end if
31:     $\beta \leftarrow \text{MIN}(\beta, v)$ 
32:  end for
33:  return  $v$ 
34: end function

```

4.2.2 Keunggulan dan Efisiensi

Alpha-Beta Pruning memberikan peningkatan performa yang signifikan dibandingkan dengan algoritma Minimax standar. Dalam skenario optimal, di mana langkah-langkah terbaik selalu dievaluasi terlebih dahulu, Alpha-Beta Pruning dapat mengurangi kompleksitas waktu dari $O(b^d)$ menjadi $O(b^{d/2})$, di mana b adalah branching factor dan d adalah kedalaman pencarian.

Efisiensi pruning sangat bergantung pada urutan evaluasi node. Pengurutan yang baik dapat meningkatkan jumlah pruning yang terjadi, sementara pengurutan yang buruk dapat mengurangi efektivitas teknik ini. Oleh karena itu, heuristik pengurutan langkah menjadi faktor penting dalam implementasi Alpha-Beta Pruning yang efisien.

$$\text{Minimax tanpa pruning} : O(b^d) \tag{4.1}$$

$$\text{Alpha-Beta Pruning (optimal)} : O(b^{d/2}) \tag{4.2}$$

$$\text{Alpha-Beta Pruning (rata-rata)} : O(b^{3d/4}) \tag{4.3}$$

Bab 5

Constraint Satisfaction Problem

5.1	What is CSP?	41
5.2	Visualisai CSP	41
5.3	Variasi dari Formulasi CSP	42
5.4	Costraint Propagation & Inference . .	43
5.5	Improving Backtracking Efficiency . .	44
5.6	Interleaving Search	45
5.7	Local Search for CSP	46

5.1 What is CSP?

Teorema 5.1

Sebuah *constraint satisfaction problem* terdiri dari tiga komponen, yaitu:

X sebuah himpunan variabel, $\{X_1, \dots, X_n\}$

D sebuah himpunan domain, satu untuk setiap variabel, $\{D_1, \dots, D_n\}$

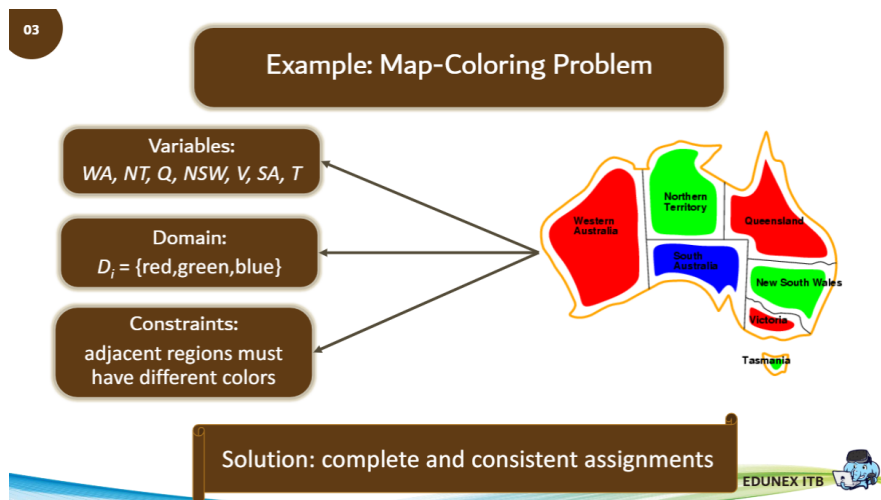
C sebuah himpunan constraints yang menentukan kombinasi yang diperbolehkan.

Konsep

CSP search algorithms memanfaatkan struktur dari states dan menggunakan *heuristics* yang bersifat general daripada *domain-specific* untuk memungkinkan penyelesaian masalah yang kompleks.

Ide utamanya adalah mengeliminasi bagian besar dari search space sekaligus dengan mengidentifikasi kombinasi *variable/value* yang melanggar *constraints*.

CSPs memiliki keuntungan tambahan bahwa *actions* dan *transition* model dapat disimpulkan dari *problem description*.



Gambar 5.1: Contoh persoalan pewarnaan peta yang dapat diselesaikan dengan CSP

5.2 Visualisai CSP

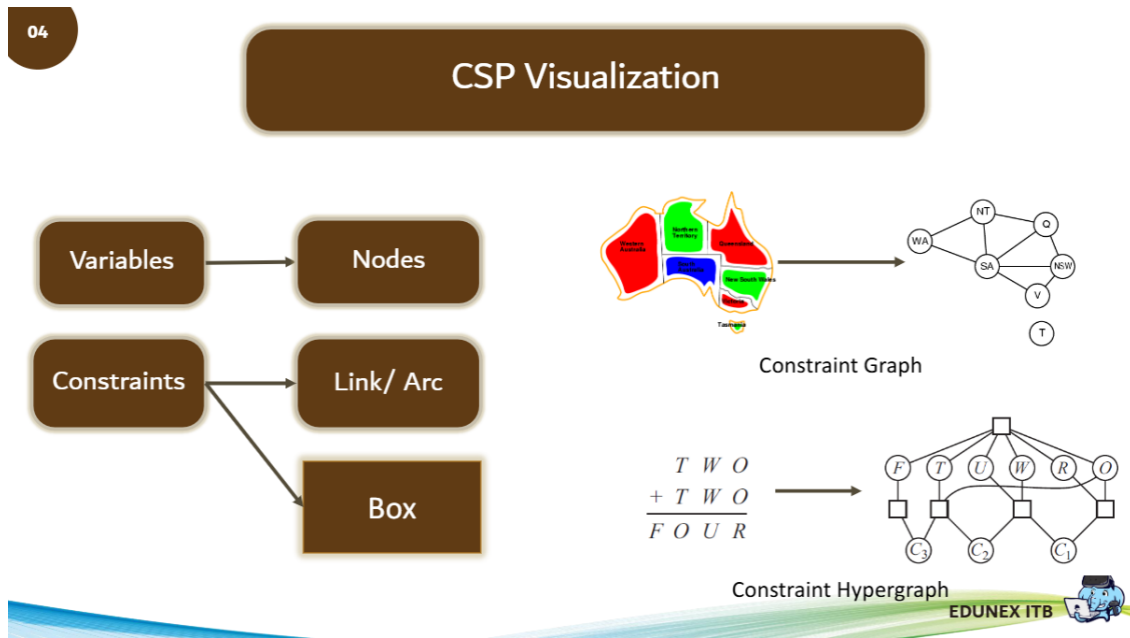
Konsep

Variabel: direpresentasikan sebagai nodes.

Constraints: direpresentasikan sebagai *link* ataupun *box* tergantung pada *graph* yang digunakan.

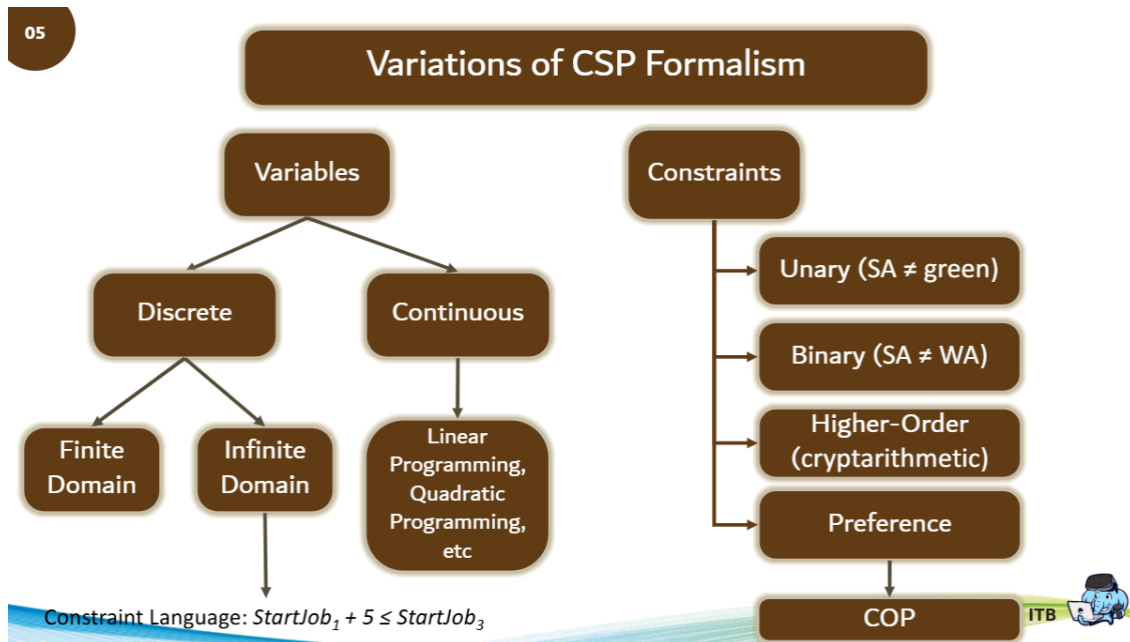
Constraint Graph: sebuah edge menghubungkan dua variables yang terlibat dalam sebuah constraint. (Lebih baik dengan jumlah *constraint* yang sedikit)

Constraint Hypergraph: sebuah box yang terhubung oleh dua nodes yang terlibat dalam sebuah constraint. (Lebih baik dengan jumlah *constraint* yang banyak)



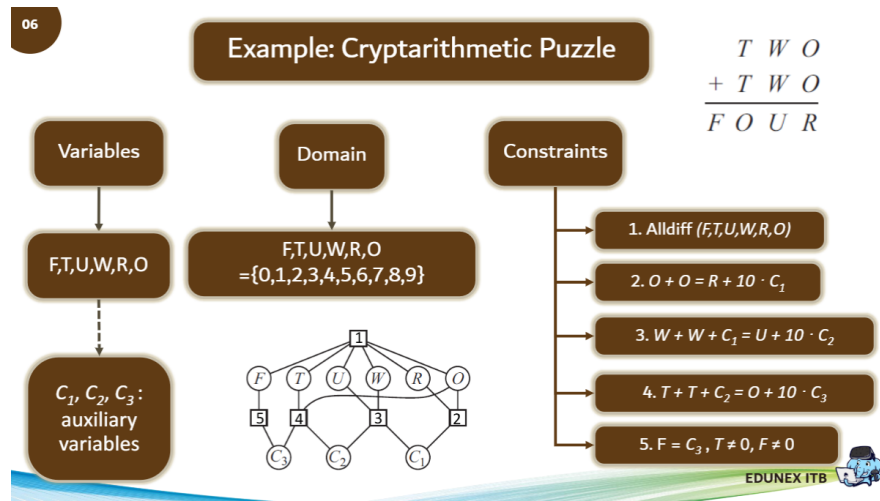
Gambar 5.2: Contoh persoalan pewarnaan peta yang dapat diselesaikan dengan CSP

5.3 Variasi dari Formulasi CSP



Gambar 5.3: Variasi dari Formulasi CSP

Contoh persoalan:



Gambar 5.4: Contoh persoalan cryptarithmic puzzle

5.4 Costraint Propagation & Inference

Teorema 5.2

Menggunakan *constraint* untuk mengurangi nilai yang diperbolehkan pada suatu variabel, yang pada akhirnya bisa mengurangi nilai yang diperbolehkan pada variabel lain.

Ide kuncinya adalah menjaga kekonsistenan lokal. Maka, prosesnya dengan menghilangkan nilai yang menyebabkan graf tersebut menjadi tidak konsisten

Konsep: Node Consistency

Jika semua nilai dalam domain variabel tersebut memenuhi batasan dari *unary constraints*.

Sebuah *network* dikatakan *node consistent* jika untuk tiap variabel yang terdapat pada *network* tersebut, variabelnya bersifat *node consistent*.

Konsep: Arc Consistency

Jika setiap nilai pada domain variabel tersebut memenuhi / tidak melanggar *binary constraint* yang ada / melibatkan variabel tersebut.

Konsep: Path Consistency

X_i, X_j adalah path-consistent terhadap X_m jika:

Assignment $X_i = a, X_j = b$ konsisten dengan constraints pada X_i, X_j .

Ada assignment ke X_m yang memenuhi constraints pada X_i, X_m dan X_m, X_j .

Konsep: K-Consistency

Sebuah CSP adalah k -consistent jika: untuk setiap himpunan dari $k - 1$ variabel dan setiap *consistent assignment* pada variabel tersebut, ada sebuah *consistent value* yang dapat diberikan pada variabel ke- k .

1-consistency: diberikan empty set, dapat membuat himpunan dari satu variabel menjadi consistent.

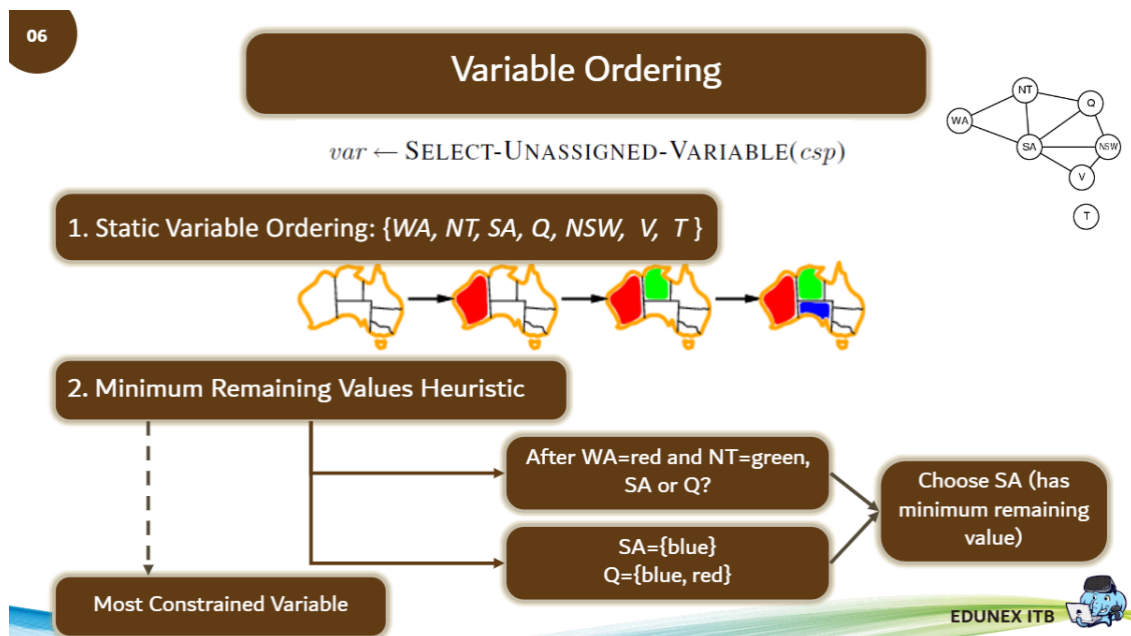
2-consistency = Arc Consistency.

3-consistency = Path Consistency.

5.5 Improving Backtracking Efficiency**Konsep: Variable Ordering**

1. **Static:** Menetapkan dari awal urutan yang diinginkan.
2. **Minimum Remaining Values Heuristic:** Memilih variabel dengan nilai kemungkinannya lebih sedikit untuk urutan selanjutnya.

Degree Heuristic: Memilih variabel dengan nilai kemungkinannya lebih sedikit untuk urutan selanjutnya.

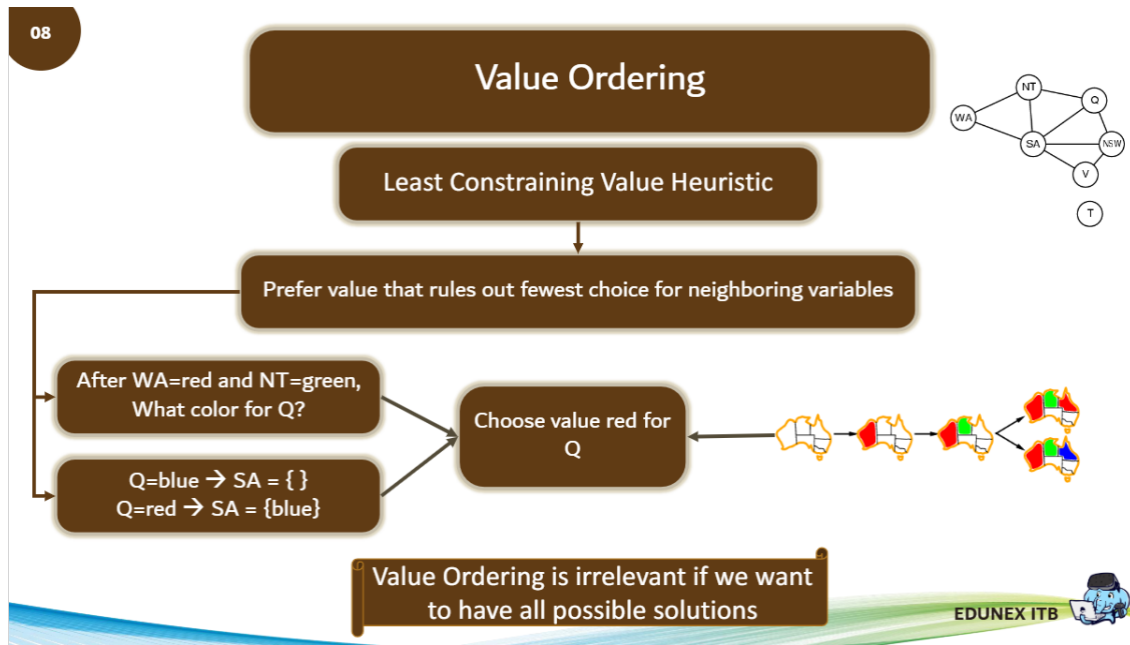


Gambar 5.5: Variable Ordering

Konsep: Value Ordering

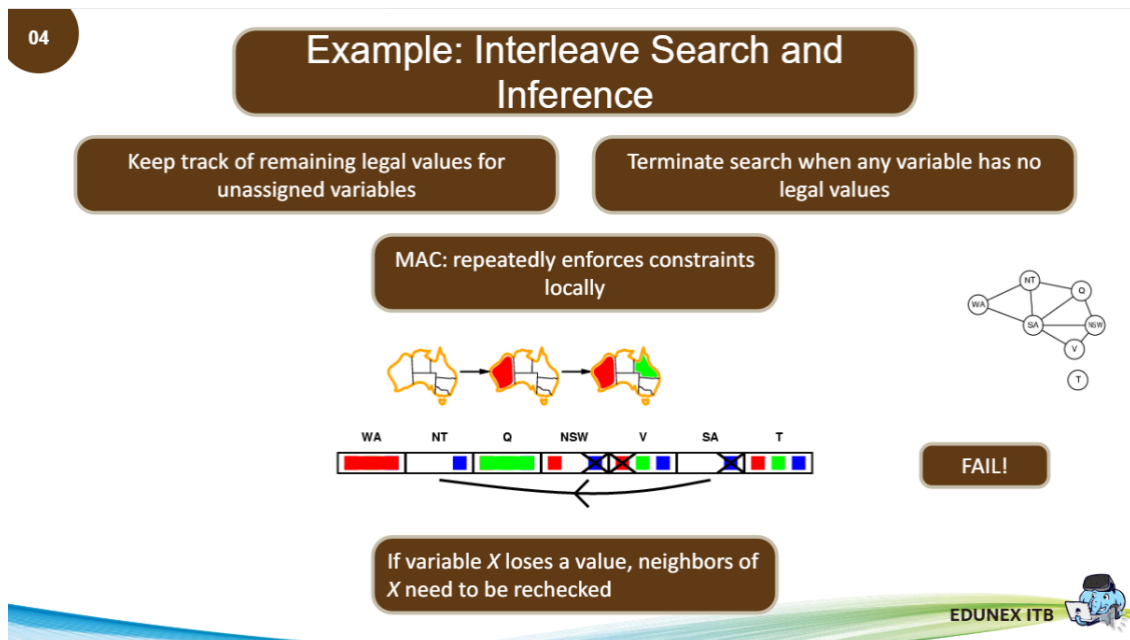
Least Constraining Value Heuristics - Lebih memilih nilai yang akan memberikan kebebasan yang lebih besar kepada variabel berikutnya yang belum di-assign nilai.

Tidak relevan ketika menginginkan semua kemungkinan solusi untuk sebuah persoalan CSP



Gambar 5.6: Value Ordering

5.6 Interleaving Search



Gambar 5.7: Value Ordering

5.7 Local Search for CSP

Konsep: Min-Conflict Heuristic

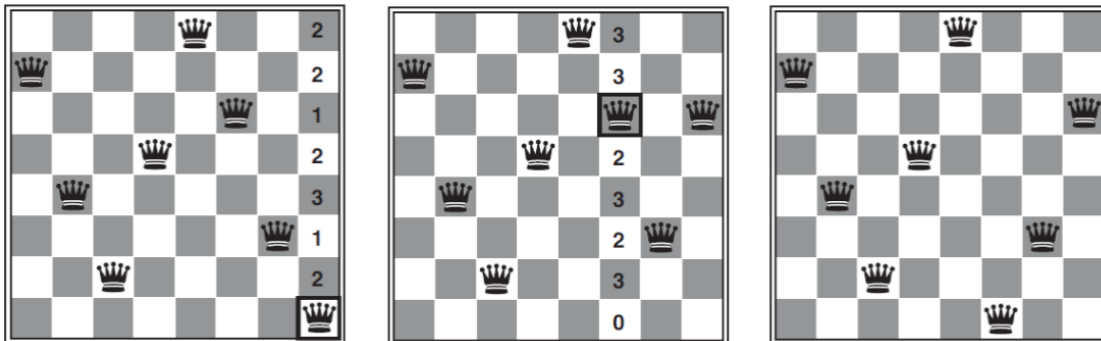
Merupakan strategi pencarian lokal untuk menyelesaikan Constraint Satisfaction Problem (CSP) dengan cara memilih variabel yang bermasalah dan mengganti nilainya agar meminimalkan jumlah konflik. Heuristik ini efisien pada masalah besar karena langsung berfokus mengurangi pelanggaran, bukan mengeksplorasi semua kemungkinan.

Algorithm 8 Min-Conflicts

```

1: function MIN-CONFLICTS(csp, max_steps)
2:   current  $\leftarrow$  sebuah penugasan lengkap awal untuk csp
3:   for i  $\leftarrow$  1 to max_steps do
4:     if current adalah solusi untuk csp then
5:       return current
6:     end if
7:     var  $\leftarrow$  pilih secara acak variabel yang konflik dari csp.VARIABLES
8:     value  $\leftarrow$  nilai v untuk var yang meminimalkan CONFLICTS(var, v, current, csp)
9:     set var  $\leftarrow$  value dalam current
10:  end for
11:  return failure
12: end function

```



Gambar 5.8: Min-Conflict Heuristic pada persoalan N-queen

Bab 6

Knowledge-based System

6.1	What is KBS?	48
6.2	KBS: Why?	49
6.3	KBS vs Conventional Program	49
6.4	Knowledge-based Agent	50
6.5	Knowledge Representation (Short Review)	51
6.6	KBS Architecture	52
6.7	Knowledge Representation (Cont.)	55
6.8	Knowledge Engineering	56

6.1 What is KBS?

Konsep

Knowledge-based System adalah sebuah sistem yang menerapkan pengetahuan (*knowledge*) untuk menyelesaikan suatu *task*.

Knowledge-based expert system: Sistem yang secara spesifik menerapkan pengetahuan untuk memecahkan masalah.

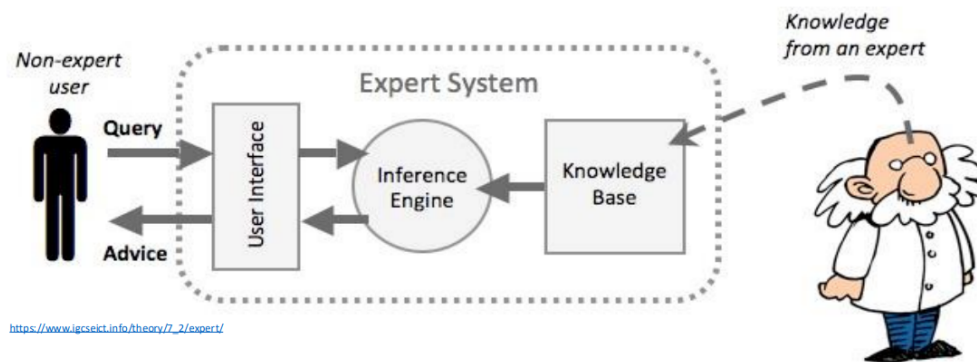
Expert System (ES): Jenis KBS yang lebih spesifik, dirancang untuk merekonstruksi keahlian (*expertise*) dan kemampuan penalaran (*reasoning*) dari seorang **spesialis** manusia dalam **domain yang terbatas**. Tujuannya adalah memberikan “saran” atau “nasihat” kepada pengguna non-ahli.

Knowledge-based agent: Sebuah agen cerdas yang menggunakan penalaran logis (*logical reasoning*) untuk bertindak di lingkungannya.

Knowledge-based System memiliki dua komponen utama dan UI:

1. **Knowledge-base:** berisi kumpulan pengetahuan yang umumnya diakuisisi dari seorang pakar untuk domain persoalan dunia nyata. Namun, pengembangannya dapat berupa sumber pengetahuan lain seperti buku, aturan umum, ataupun kumpulan kasus yang pernah diselesaikan.
2. **Inference Engine:** “Otak” yang melakukan penalaran berdasarkan pengetahuan yang ada.
3. **User Interface:** Jembatan antara pengguna non-ahli dan sistem, di mana pengguna bisa bertanya (query) dan sistem memberikan nasihat (advice).

Knowledge-based System $\neq$ Expert System



Gambar 6.1: Komponen dari Knowledge-based System

Notes

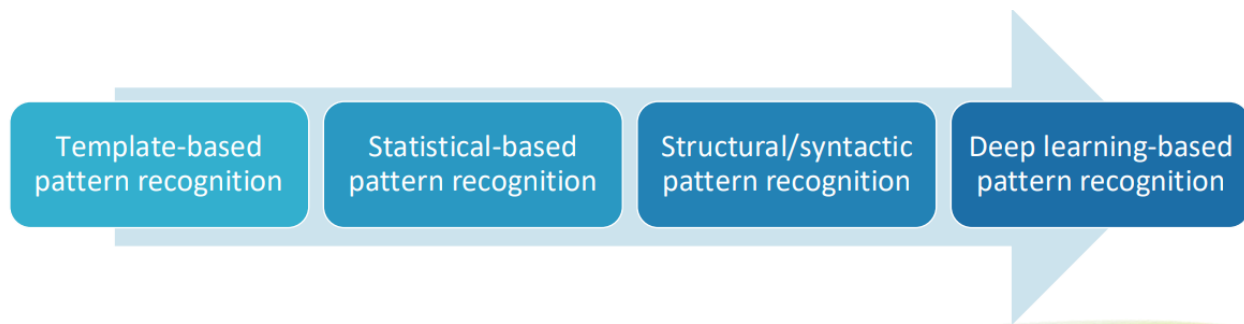
Semua *knowledge-based system* **belum tentu** merupakan *expert system* tergantung dari narasumbernya. Akan tetapi, semua *expert system* merupakan *knowledge-based system* yang narasumbernya adalah pakar (*expert*).

6.2 KBS: Why?

KBS merupakan salah satu pendekatan utama dalam pengembangan agen AI, khususnya yang berfokus pada penalaran logis (*logical reasoning*) atau “*thinking rationally*”.

Terdapat empat pendekatan pada *pattern recognition* secara umum yang semuanya merupakan KBS, mulai dari paling sederhana hingga kompleks:

1. **Template-based:** *Knowledge-base* berupa kumpulan *template* seperti karakter pada sistem pengenalan karakter tulisan, lalu inferensinya berupa *template matching*
2. **Statistical-based:** *Knowledge-base* berupa model hasil pembelajaran, lalu inferensinya sesuai dengan representasi model tersebut.
3. **Structural/syntactic:** *Knowledge-base* berupa struktur yang dibangun oleh sejumlah fitur, lalu inferensinya menelusuri struktur tersebut.
4. **Deep Learning:** *Knowledge-base* berupa model Artificial Neural Network (ANN) dengan *layer* yang banyak, lalu inferensinya melakukan *forward propagation* untuk ANN.



Gambar 6.2: Empat pendekatan pada *pattern recognition*

6.3 KBS vs Conventional Program

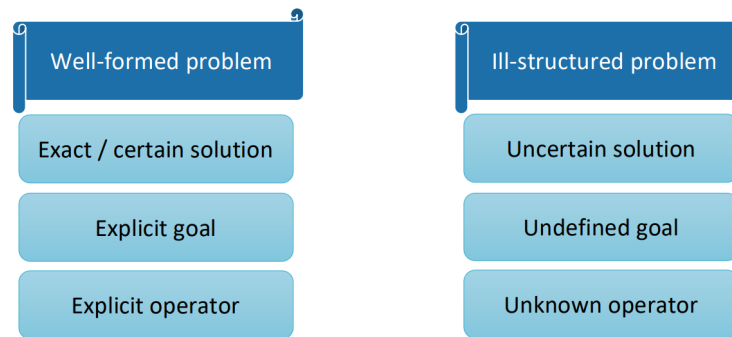
Perbedaan paling mendasar antara KBS dan program konvensional terletak pada jenis masalah yang ditangani dan cara kerjanya.

Fitur	Knowledge-Based System (KBS)	Program Konvensional
Jenis Masalah	<i>Ill-structured problem</i> (masalah tidak terstruktur).	<i>Well-structured problem</i> (masalah terstruktur).
Penentu Aksi	Pakar menentukan aksi, namun urutan eksekusi ditentukan oleh <i>interpreter</i> (inference engine) saat runtime.	Programmer menentukan aksi dan urutan eksekusinya secara eksplisit.
Komponen	Metode pemecahan masalah + Pengetahuan domain + Data.	Algoritma + Data.

Tabel 6.1: Perbandingan Knowledge-Based System dan Program Konvensional

Pada KBS, ada pemisahan antara “pengetahuan” (apa yang diketahui sistem) dan “mesin inferensi” (bagaimana sistem berpikir), membuatnya lebih fleksibel.

Problem Characteristics



Gambar 6.3: Penjelasan lanjut terkait perbedaan jenis masalah

6.4 Knowledge-based Agent

Mengingat kembali pada studi kasus Wumpus World pada Bab 2: "Agent".

Konsep

Knowledge-Based Agent (KBA) adalah agen yang menggunakan properti dari *logical reasoning* (*first order logic* dan *formal logic*) untuk bertindak di lingkungannya.

Agen ini memiliki sebuah *Knowledge-base* (KB) atau basis pengetahuan yang berisi informasi tentang dunia.

```

function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
               t, a counter, initially 0, indicating time

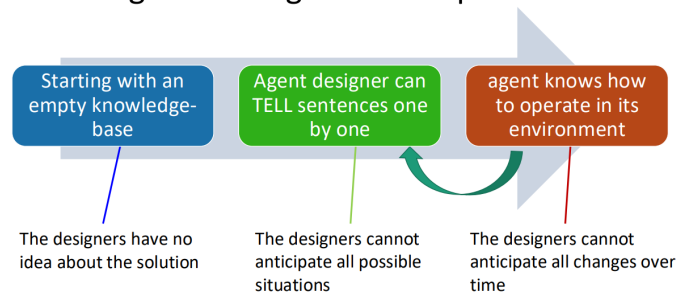
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t)) {assert percept}
  action ← ASK(KB, MAKE-ACTION-QUERY(t)) {reasoning}
  TELL(KB, MAKE-ACTION-SENTENCE(action, t)) {assert action}
  t ← t + 1
  return action
  
```

Gambar 6.4: Generic Knowledge-based Agent

Properti fundamentalnya adalah:

- Dalam setiap langkah, agen menarik kesimpulan dari informasi yang tersedia di KB.
- Kesimpulan tersebut dijamin benar jika informasi yang tersedia di KB juga benar.
- Mulai dari *knowledge-base* **kosong**
- Mengisi *knowledge-base* **sentence by sentence**
- Agent jadi tau gimana cara beroperasi di lingkungannya

Knowledge-based Agent Development



Gambar 6.5: Workflow Knowledge-based Agent Development

6.5 Knowledge Representation (Short Review)

Konsep

Knowledge Representation adalah sebuah bahasa formal untuk merepresentasikan pengetahuan atau informasi.

Bahasa ini memiliki dua komponen utama:

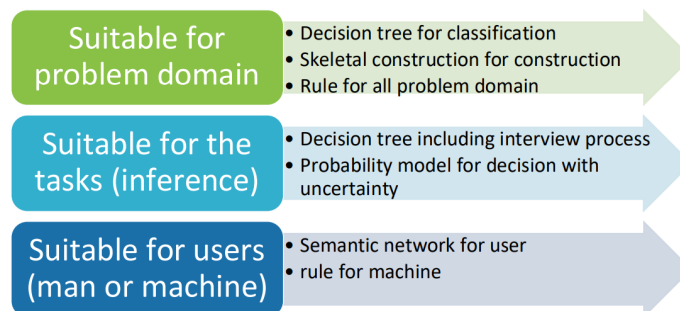
Sintaks (Syntax): Aturan yang mendeskripsikan ekspresi apa yang legal atau valid untuk ditulis dalam bahasa tersebut. Contoh: Dalam logika proposisional, $\wedge \wedge Q$ adalah sintaks yang valid, sedangkan $P \wedge \wedge Q$ tidak.

Semantik (Semantic): Cerita atau interpretasi yang menjelaskan makna dari ekspresi-ekspresi tersebut. Contoh: Semantik dari $P \wedge Q$ adalah “proposisi P benar DAN proposisi Q benar.”

Agar efektif, sebuah KR harus memenuhi beberapa syarat:

1. Tidak ada kontradiksi: Representasi tidak boleh mengandung informasi yang saling bertentangan.
2. Setiap simbol harus unik: Setiap simbol harus memiliki makna yang jelas dan tunggal.
3. Mampu menjelaskan objek, relasi, dan atribut: Harus bisa mendeskripsikan dunia dengan cukup detail.
4. Manipulasi yang efisien: Harus bisa diproses oleh komputer secara efisien.

Memilih metode KR yang tepat sangat bergantung pada konteks masalah dan pengguna.



Gambar 6.6: Pemilihan metode *Knowledge Representation*

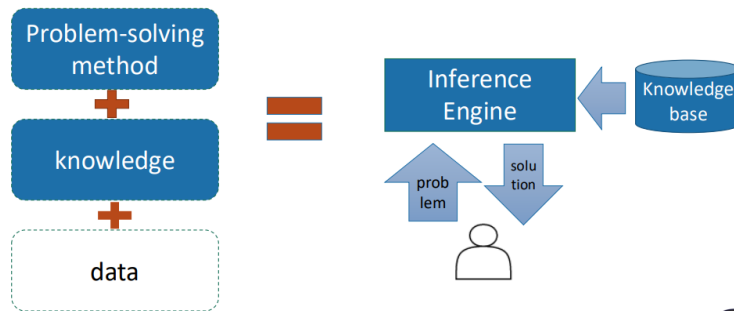
6.6 KBS Architecture

Teorema 6.1

KBS merupakan gabungan dari

Problem-solving method + knowledge + data

Problem-solving method merupakan algoritma yang menentukan bagaimana *domain-specific knowledge* digunakan untuk menyelesaikan persoalan (Puppe, 1993). Sebagai contoh, *knowledge reasoning methods* (*forward chaining* untuk *rule*) dan *general procedures* (*partially order plan*).

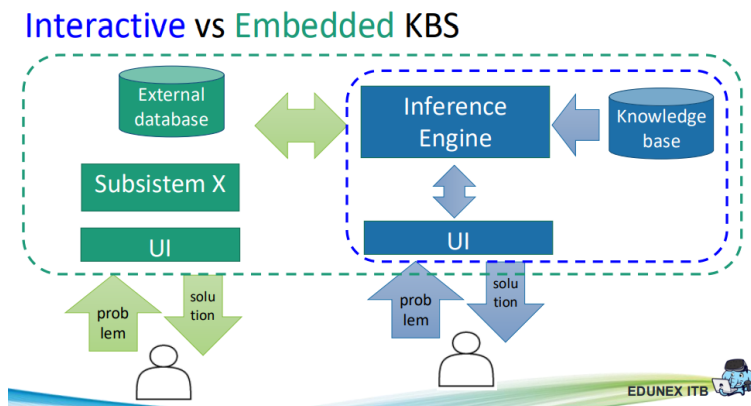


Gambar 6.7: Terminology pada KBS

Problem-solving method diimplementasikan pada *inference engine*, sedangkan *knowledge* disimpan pada *knowledge-base*. *User* akan memberikan data berupa *problem* kepada *inference engine* dan menerima solusinya.

Terdapat dua jenis KBS secara umum:

1. **Interactive:** *User* langsung berhubungan dengan KBS melalui *User Interface* yang ada.
2. **Embedded:** KBS berinteraksi dengan subsistem yang lain ataupun mengakses *external database* dari sistem yang lebih besar. Jika diperlukan interaksi, sistem yang lebih besar akan menyediakan *User Interface*.



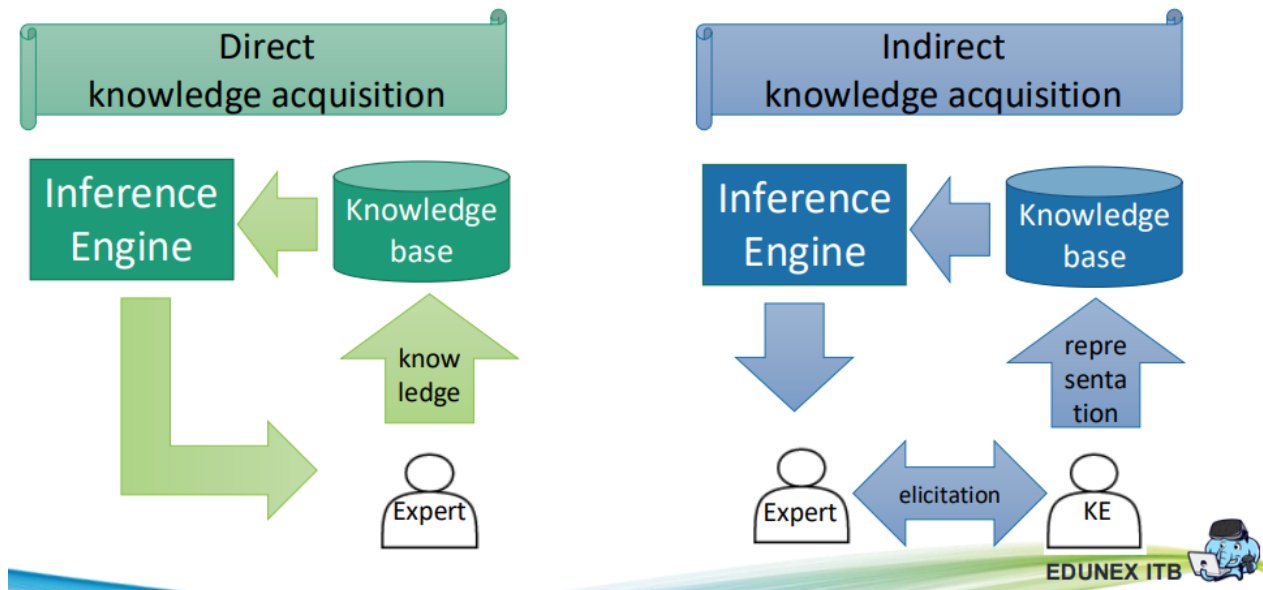
Gambar 6.8: Interactive dan Embedded KBS

Konsep: Knowledge Acquisition

Knowledge Acquisition merupakan istilah untuk proses mengekstraksi pengetahuan pakar dan mengonversinya ke dalam suatu bentuk yang dapat digunakan oleh komputer untuk menyelesaikan persoalan.

Terdapat dua jenis *knowledge acquisition*:

1. **Direct:** Pakar melakukan formulisasi dan strukturisasi pengetahuannya **secara langsung**, biasanya didukung oleh lingkungan akuisisi pengetahuan dengan **fasilitas tambahan**, seperti memilih indikator dari kasus yang mungkin terjadi dan menentukan solusinya. Hasil dari KBS tetap diberikan ke pakar kembali untuk verifikasi.
2. **Indirect:** Terdapat *knowledge engineer* yang akan melakukan ekstraksi atau elisitasi pengetahuan domain dan *strategic* dari pakar **dalam bentuk dialog atau interview** yang berulang. *Knowledge engineer* melakukan representasi dalam bentuk program deklaratif untuk mengisi *knowledge-base*, lalu pengujiannya tetap diberikan ke pakar kembali.



Gambar 6.9: Interactive dan Embedded KBS

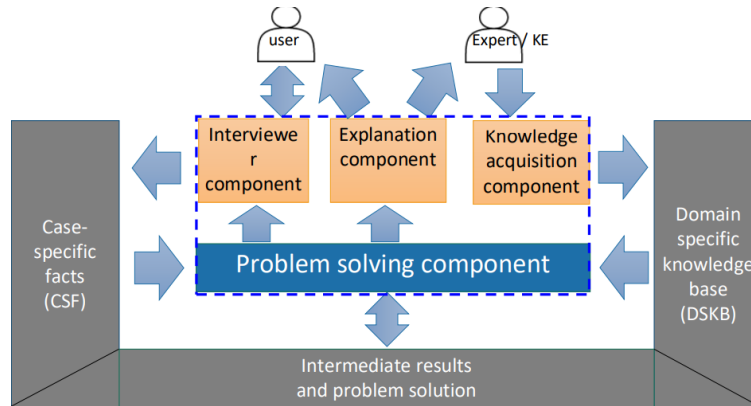
Terdapat dua modul utama pada arsitektur umum dari *knowledge-based system*, yaitu *knowledge-base* dan *control system*.

Knowledge-base terdiri atas tiga bagian:

1. **Case-specific facts (CSF):** Menyimpan fakta-fakta untuk tiap sesi interaksi.
2. **Intermediate result:** Menyimpan hasil sementara inferensi.
3. **Domain-specific knowledge base:** Pengetahuan domain.

Lebih lanjut, *control system* dari atas dua bagian:

1. **Problem-solving component:** sebagai mesin inferensi.
2. **User interface:** berisi submodul *interviewer*, *explanation*, *knowledge acquisition*.



Gambar 6.10: Arsitektur Umum KBS

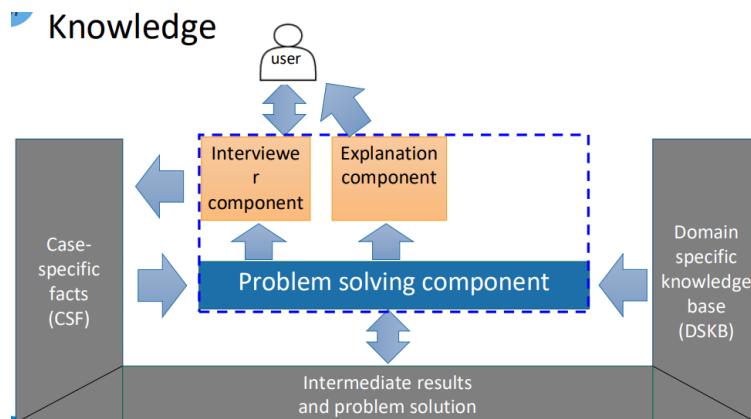
Ketika *user* interaksi dengan KBS melalui *interviewer component* maka fakta-fakta disimpan pada *case-specific facts*. Kemudian, fakta tersebut dilakukan *matching* oleh *problem-solving component* (mesin inferensi) dan hasil inferensinya disimpan pada *working memory* yang disebut *intermediate results*.

Setelah didapatkan *problem solution* secara iteratif, hasilnya diberikan *interviewer component* kepada *user*. Jika *user* ingin mendapatkan dari hasil yang diberikan, *problem-solving component* akan menjelaskan proses inferensinya melalui *explanation component* ataupun *expert* yang *monitoring* KBS kepada *user*.

Expert dan *knowledge engineer* bisa meng-*update* pengetahuan dari KBS melalui *knowledge acquisition component* dan pengetahuannya akan disimpan pada *domain-specific knowledge base*.

Interactive, No Update Knowledge

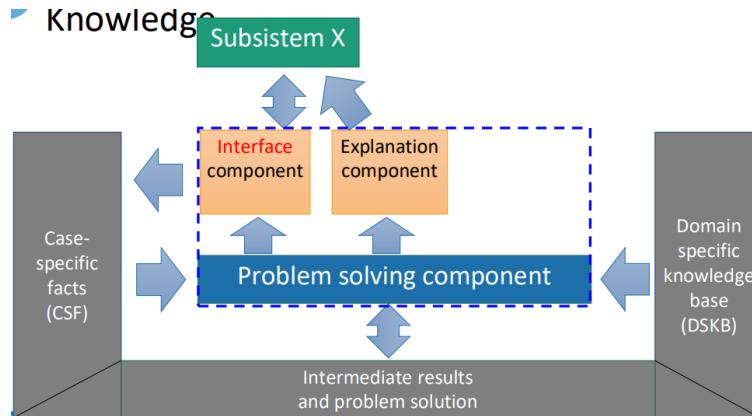
Ketika ingin mendesain suatu KBS yang interaktif tanpa memerlukan *update* pengetahuan, artinya pakar tidak akan lagi memasukkan pengetahuan. Maka, dapat dihapus *knowledge acquisition component* dan peran pakar.



Gambar 6.11: Interactive, No Update Knowledge

Embedded, No Update Knowledge

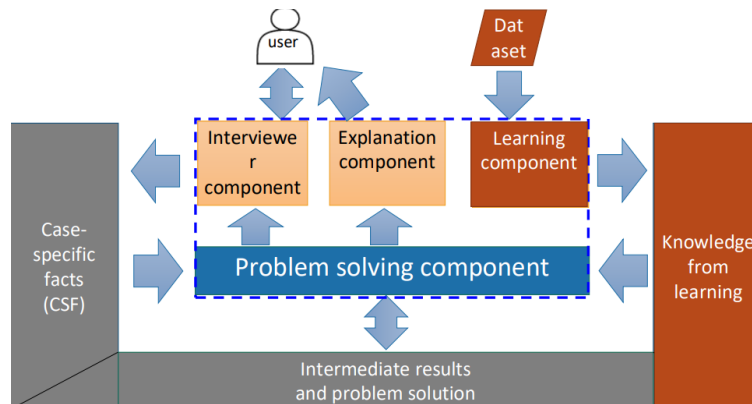
Ketika ingin mendesain suatu KBS yang *embedded* pada sistem yang lebih besar dan tanpa memerlukan *update* pengetahuan. Maka, tidak ada *user* yang berinteraksi langsung dan KBS akan berinteraksi dengan subsistem.



Gambar 6.12: Embedded, No Update Knowledge

Embedded, No Update Knowledge

Ketika ingin mendesain suatu KBS yang interaktif dan *update* pengetahuan dilakukan melalui *learning* dari data. Maka, diubah *knowledge acquisition component* menjadi *learning component* yang akan belajar dari *dataset* yang disediakan sehingga *domain specific knowledge base* menjadi *knowledge from learning*.



Gambar 6.13: Interactive, Learning

6.7 Knowledge Representation (Cont.)

Terdapat berbagai bentuk yang umum dari *knowledge representation*:

1. Propositional and Predicate Logic:

Sistem berbasis pengetahuan yang menggunakan logika proposisional atau predikat untuk merepresentasikan pengetahuan umumnya melibatkan aturan IF-THEN, hubungan antar objek dan kuantifikasi objek. Contoh: Smart Building Control System dan Automated Theorem Proving.

2. Semantic Networks (Ontology):

Semantic Networks merepresentasikan pengetahuan dalam bentuk graf, di mana konsep atau objek diwakili sebagai node, dan hubungan di antara konsep tersebut diwakili sebagai edge. Contoh: Google Knowledge Graph dan PayPal Fraud Detection System.

3. Frames:

Frames adalah struktur data yang digunakan untuk merepresentasikan pengetahuan dalam bentuk slot (tempat penyimpanan informasi) dan filler (nilai atau informasi yang mengisi slot). Contoh: Chatbot Asisten Virtual dan Robotic Navigation Systems.

4. Production Rule:

Production rule adalah representasi pengetahuan dalam bentuk aturan berbasis kondisi dan aksi, atau sering disebut dengan if-then rules. Contoh: sistem pakar medis MYCIN dan sistem deteksi intrusi Snort.

5. Case-Based Reasoning:

Case-Based Reasoning (CBR) adalah metode representasi pengetahuan di mana solusi dari masalah-masalah sebelumnya (kasus) digunakan kembali untuk memecahkan masalah baru yang serupa. Penalaran ini bergantung pada basis data kasus yang mencakup solusi atau keputusan yang diambil dalam skenario sebelumnya. Contoh: Case-Based Radiology System dan HelpDesk Systems.

6. Bayes Network:

Berpikir dalam ketidakpastian. Mewakili ketidakpastian tidak hanya melalui disjungsi (Logika), tetapi juga kemungkinan (probabilitas).

6.8 Knowledge Engineering

Knowledge Acquisition (KA) dalam suatu domain dari satu atau lebih sumber non-elektronik dan konversinya ke dalam suatu bentuk yang dapat digunakan oleh komputer untuk memecahkan persoalan yang umumnya hanya dapat dipecahkan oleh pakar domain tersebut.

$$KA = \text{knowledge elicitation} + \text{representation}$$

Dalam hal ini, *knowledge elicitation* merupakan Proses ekstraksi pengetahuan domain dan strategik dari pakar, seperti *interview* antara KE dan pakar. *Knowledge representation* merupakan proses merepresentasikan pengetahuan hasil ekstraksi ke suatu bentuk formal.

Terdapat lima tahapan *knowledge acquisition*:

1. **Identification:** Identifikasi karakteristik masalah
2. **Conceptualization:** Menemukan konsep-konsep untuk merepresentasikan pengetahuan
3. **Formalization:** Design struktur untuk mengorganisasikan pengetahuan
4. **Implementation:** Formulasi pengetahuan ke bentuk *runnable* program
5. **Testing:** Validasi pengetahuan

Kemudian, terdapat dua teknik *knowledge acquisition*:

1. **Manual:** Dapat melalui *interview*, observasi, dan intuitif (tukar peran *knowledge engineer* dan pakar)
2. **Otomatis:** Menggunakan *tools* untuk memfasilitasi akuisisi, seperti *tools* untuk pakar atau *machine learning*

Bab 7

Rule-based System

7.1	RBS: What and Why?	58
7.2	Rule: Logical Implication	58
7.3	Arsitektur RBS	59
7.4	Metode Inferensi (Penalaran)	59
7.5	Forward Chaining	60
7.6	Backward Chaining	63
7.7	Advantages of RBS	65

7.1 RBS: What and Why?

Konsep

Rule-Based System (RBS) adalah salah satu jenis Knowledge-Based System (KBS) di mana pengetahuan direpresentasikan dalam bentuk **aturan-aturan** atau **rules**.

$$\text{Rule} = \text{precondition} - \text{action}$$

Akibat kesederhanaan dan kekuatannya, ini adalah salah satu solusi AI yang paling umum dan tersebar luas di dunia nyata.

Alasan Penggunaan RBS:

1. **Representasi Pengetahuan Paling Sederhana:** Aturan IF-THEN adalah cara yang sangat intuitif dan mudah dipahami oleh manusia (terutama pakar domain) untuk mengekspresikan pengetahuan.
2. **Solusi Paling Umum:** Banyak masalah di dunia nyata, seperti diagnosis, konfigurasi, dan penjadwalan, dapat dimodelkan dengan baik menggunakan aturan.
3. **Pendekatan Hybrid:** RBS sering digabungkan dengan teknik lain seperti Machine Learning (RBS+ML) untuk menciptakan sistem yang lebih kuat, di mana RBS menangani penalaran tingkat tinggi dan ML menangani pengenalan pola.

7.2 Rule: Logical Implication

Sebuah rule pada dasarnya adalah implementasi dari implikasi logis, yang memiliki format dasar IF-THEN:

IF (kondisi-kondisi tertentu terpenuhi) THEN (lakukan aksi-aksi berikut).

1. **IF Part (LHS):** Bagian ini disebut Left-Hand Side (LHS), premis, atau prekondisi. Ini berisi serangkaian kondisi yang harus dipenuhi.
2. **THEN Part (RHS):** Bagian ini disebut Right-Hand Side (RHS), konklusi, atau aksi. Bagian ini akan dieksekusi jika semua kondisi di LHS terpenuhi. Aksi ini bisa berupa menambahkan fakta baru, menghapus fakta, atau menjalankan suatu prosedur.

IF *certain conditions are true* — Preconditions, premises, LHS,
 THEN *execute the following actions* — Actions, conclusion, RHS

CLIPS: C Language Integrated Production System

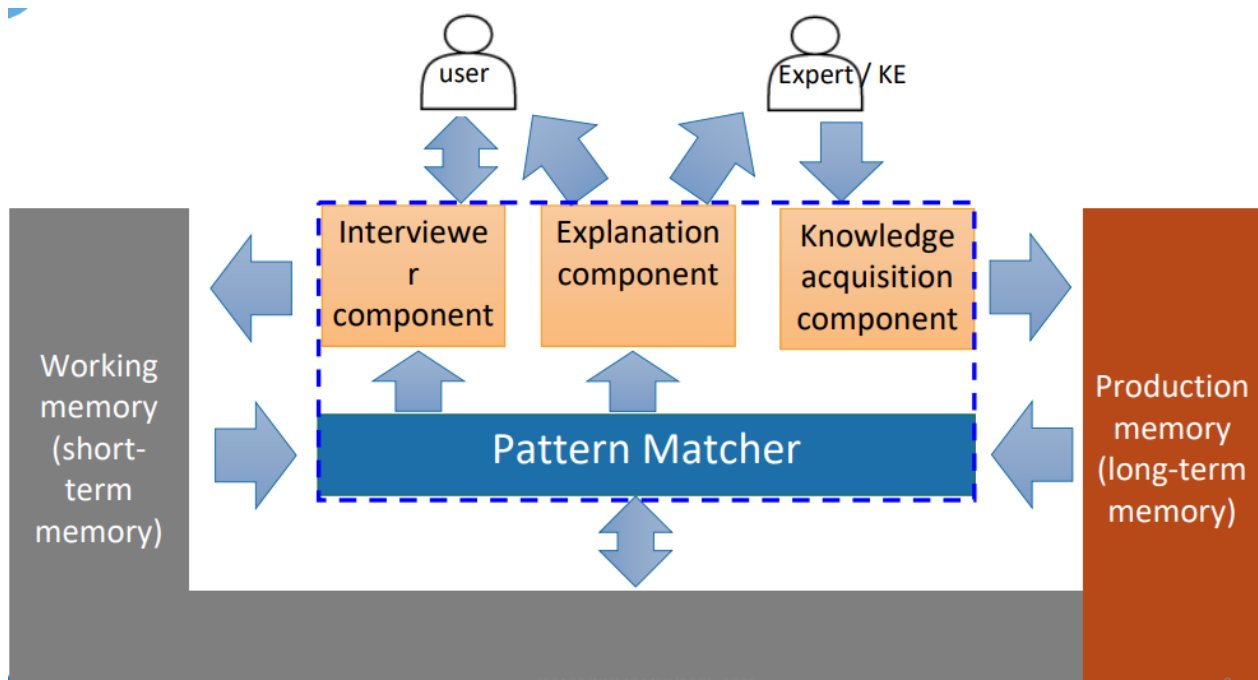
```
(defrule R
  (is-a ?x horse)
  (is-parent-of ?x ?y)
  (is-fast ?y)
=>
  (assert (is-valuable ?x))
)
```

Gambar 7.1: Komponen dari Rule

7.3 Arsitektur RBS

RBS merupakan KBS maka arsitektur umumnya sama. Sebuah RBS umumnya terdiri dari beberapa komponen inti:

1. **Production Memory:** Menyimpan kumpulan *rules*, bersifat statis.
2. **Working Memory:** Menyimpan fakta dan hasil inferensi, bersifat dinamis.
3. **Pattern Matcher:** Melakukan pencocokan pola antara fakta dan prekondisi pada *rules*



Gambar 7.2: Arsitektur Umum RBS

7.4 Metode Inferensi (Penalaran)

Ada dua cara utama bagi RBS untuk melakukan penalaran:

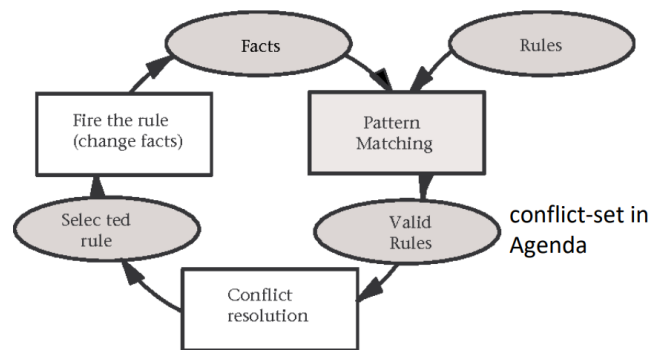
1. **Forward Chaining (Data-Driven):** Penalaran dimulai dari fakta-fakta yang ada untuk mencapai sebuah kesimpulan. Sistem akan terus mengeksekusi aturan yang kondisinya terpenuhi oleh fakta saat ini (*LHS matching*).
2. **Backward Chaining (Goal-Driven):** Penalaran dimulai dari sebuah tujuan atau hipotesis. Sistem mencoba membuktikan tujuan tersebut dengan mencari aturan yang bisa menghasilkan tujuan itu, lalu bekerja mundur untuk membuktikan kondisi-kondisi dari aturan tersebut (*RHS matching*).

7.5 Forward Chaining

Konsep

Forward Chaining adalah metode inferensi (penalaran) dalam RBS yang bekerja secara *data-driven*. Artinya, proses dimulai dari sekumpulan fakta yang sudah ada di *Working Memory* untuk kemudian mencari dan mengeksekusi aturan-aturan yang relevan hingga mencapai sebuah kesimpulan atau tujuan. Proses ini sering juga disebut Siklus Kenali-Bertindak (*Recognize-Act Cycle*).

Forward Chaining: Recognize-Act Cycle



Gambar 7.3: Forward Chaining

Diawali dengan proses *pattern matching* dari fakta dan *rules* dilakukan *matching*. Kemudian, semua *rules* yang pre kondisinya terpenuhi oleh fakta akan menghasilkan *valid rules* atau *conflict-set* yang disimpan pada *working memory* khusus yang disebut dengan Agenda. Setelah itu, diproses oleh *conflict resolution* untuk menentukan *rule* yang akan dipilih. *Rule* yang terpilih akan dilakukan *fire the rule* yang kemudian meng-*update* fakta pada *working memory*. Iterasi akan lanjut dilakukan mulai dari *matching* kembali sampai kondisi yang diinginkan terpenuhi atau solusi sudah tercapai.

Algorithm 9 Forward Chaining

```

1: function FORWARDCHAINING(initial_facts)
2:   data ← initial_facts
3:   repeat
4:     conflictSet ← ∅
5:     for all rule R in rule base do
6:       if preconditions of R are satisfied by data then
7:         conflictSet ← conflictSet ∪ {R}
8:       end if
9:     end for
10:    if conflictSet = ∅ then
11:      return data
12:    end if
13:    R ← select a rule from conflictSet by conflict-resolving strategy
14:    data ← result of applying action part of R to data
15:  until termination condition is satisfied
16:  return data
17: end function

```

▷ No more rules to apply

Strategi Resolusi Konflik (Conflict-Resolution Strategy)

Strategi ini sangat penting untuk menentukan perilaku sistem. Strategi ini dibagi menjadi dua kategori, yaitu *Global Control* (Strategi Umum) dan *Local Control* (Strategi Khusus).

Strategi *Global Control* berlaku secara umum untuk semua aturan.

Konsep: Refractoriness

Strategi paling dasar. Sebuah aturan tidak akan dieksekusi lebih dari satu kali dengan set fakta yang persis sama. Ini mencegah sistem masuk ke dalam infinite loop yang trivial.

Refractoriness

Do not select a rule that has just been applied with the same values of its variables (Brachman, 2004).

Rule-base:

R1: IF (lecturing X) AND (marking-practicals X) THEN ADD (overworked X)
 R2: IF (month february) THEN ADD (lecturing alison)
 R3: IF (month february) THEN ADD (marking-practicals alison)
 R4: IF (overworked X) OR (slept-badly X) THEN ADD (bad-mood X)
 R5: IF (bad-mood X) THEN DELETE (happy X)
 R6: IF (lecturing X) THEN DELETE (researching X)

Facts:

(month february)
 (happy alison)
 (researching alison)



Iteration	CS	R
1	{R2, R3}	R2
2	{ R2 , R3, R6}	R3
3	etc....	

Gambar 7.4: Refractoriness dalam Forward Chaining

Konsep: Selection by Order

Rule Order (FIFO): Aturan yang pertama kali masuk ke *Conflict Set* yang akan dieksekusi. Urutan aturan di *Knowledge Base* menjadi penting.

Fact Recency (LIFO): Aturan yang dipicu oleh fakta yang paling baru ditambahkan ke *Working Memory* yang akan dieksekusi. Ini membuat sistem lebih responsif terhadap perubahan terbaru.

Knowledge-base:

R1: **if** (priority second)
then out("print second")
 R2: **if** (priority first)
then out("print first")
 R3: **if** (priority third)
then out("print third")

Facts:

(priority first)
 (priority second)
 (priority third)

➡ • Selection by rule order (FIFO):
 print second
 print first
 print third

➡ • Selection by fact (recency) order (LIFO):
 print third
 print second
 print first

Gambar 7.5: Selection by Order dalam Forward Chaining

Konsep: Specificity

Memilih aturan yang paling spesifik. Aturan yang memiliki lebih banyak kondisi di bagian LHS-nya dianggap lebih spesifik dan akan diprioritaskan. Tujuannya adalah mendahulukan kasus-kasus khusus sebelum menangani kasus umum.

- Specificity: select **most specific rule** first
- Example:
 - Conflict set: {R1,R2}
 - R1: if A, B, C then <aksi R1>
 - R2: if A,C then <aksi R2>
 - A and B and C is more specific than A and C → select R1

Gambar 7.6: Specificity dalam Forward Chaining

Lebih lanjut, Strategi *Local Control* memberikan kontrol lebih kepada perancang sistem.

1. **Selection by Priority:** Setiap aturan diberi bobot prioritas (misalnya, salience di CLIPS). Aturan dengan prioritas tertinggi dalam Conflict Set yang akan dieksekusi. Ini memungkinkan perancang untuk secara eksplisit menentukan aturan mana yang lebih penting.
2. **Selection by Meta-rules:** Menggunakan “aturan tentang aturan”. Ini adalah aturan tingkat lanjut yang tugasnya adalah mengontrol atau mem-*pruning* (memangkas) aturan lain dalam *Conflict Set*. Contoh: “JIKA ada dua aturan yang bertentangan, dahulukan aturan yang berasal dari sumber yang lebih terpercaya.”

Select high priority rule

Example:

R1: if (burung ?X)
 then (terbang ya)
 R2: if (burung penguin)
 (declare salience 100)
 then (terbang tidak)
 Fakta: (burung penguin)

Meta rules

Pruning rules:

if the culture was not obtained from a sterile
 source,
 there are rules which mention in their
 premise a previous organism
 then each of them is not going to be useful

Gambar 7.7: Selection by Supplementary Knowledge dalam Forward Chaining

Berikut contoh dari *forward chaining*:

Forward Chaining: Is Z true ?

Rule-base:

R1: Y, D → Z
 R2: X, B, E → Y
 R3: A → X
 R4: C → L
 R5: L, M → N

Facts:

A,B,C,D,E

Conflict resolution strategy:

refractoriness > fact recency > specificity > rule order

Iteration	Conflict set	Selected Rule	Working memory
1	{R3, R4}	R4	+ L
2	{R3, R4}	R3	+ X
3	{R2, R3, R4}	R2	+ Y
4	{R1, R2, R3, R4}	R1	+Z
5	{R1, R2, R3, R4}	-	stop

Answer: Yes

Gambar 7.8: Contoh Forward Chaining

7.6 Backward Chaining

Konsep

Backward Chaining adalah metode inferensi dalam RBS yang bekerja secara *goal-driven* (digerakkan oleh tujuan). Berbeda dengan Forward Chaining yang memulai dari fakta, Backward Chaining memulai dari sebuah hipotesis atau tujuan (*goal*) dan bekerja secara mundur untuk menemukan fakta-fakta yang mendukungnya. Proses ini sering juga disebut *Goal-Driven Reasoning*.

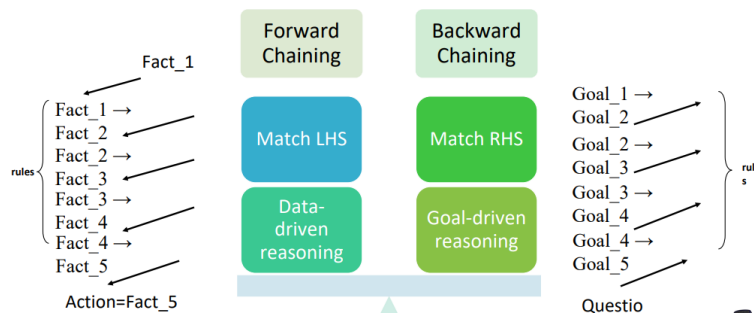
Algorithm 10 Backward Chaining

```

1: function BACKWARDCHAINING(goal, facts)
2:   if goal ∈ facts then
3:     return true
4:   end if
5:   conflictSet ← ∅
6:   for all rule R in rule base do
7:     if conclusion of R matches goal then
8:       conflictSet ← conflictSet ∪ {R}
9:     end if
10:  end for
11:  if conflictSet = ∅ then
12:    FINDOUT(goal, facts)
13:    if goal ∈ facts then
14:      return true
15:    else
16:      return false
17:    end if
18:  end if
19:  R ← select a rule from conflictSet by conflict-resolving strategy
20:  subgoals ← preconditions of R
21:  for all subgoal in subgoals do
22:    if not BACKWARDCHAINING(subgoal, facts) then
23:      return false
24:    end if
25:  end for
26:  MONITOR(R, facts)
27:  return true
28: end function

```

▷ Goal is already a known fact
 ▷ Ask user or check sensors
 ▷ Cannot prove subgoal
 ▷ Execute and monitor rule action
 ▷ All subgoals are proven



Gambar 7.9: Forward Chaining vs Backward Chaining

Untuk memudahkan pemahaman, berikut perbedaan dari Forward Chaining dan Backward Chaining:

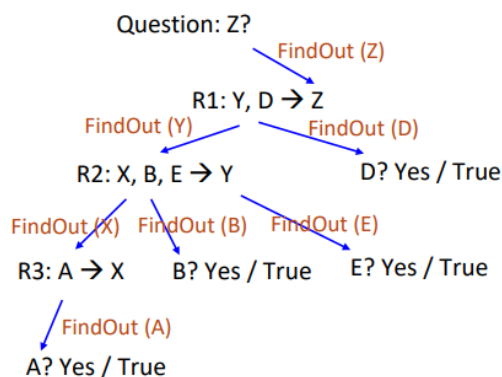
Fitur	Forward Chaining	Backward Chaining
Arah Penalaran	Dari Fakta $\rightarrow$ ke Kesimpulan	Dari Tujuan $\leftarrow$ ke Fakta
Pemicu	Data-driven (digerakkan data)	Goal-driven (digerakkan tujuan)
Pencocokan	Mencocokkan fakta dengan LHS (premis) aturan.	Mencocokkan tujuan dengan RHS (konklusi) aturan.
Pertanyaan	"Apa yang bisa disimpulkan dari fakta-fakta ini?"	"Apakah tujuan/hipotesis ini benar?"

Tabel 7.1: Perbandingan Forward Chaining dan Backward Chaining

Alur Kerja Backward Chaining

Alur kerjanya dapat diibaratkan seperti seorang detektif yang memiliki hipotesis ("Tersangka X pelakunya") dan harus mencari bukti-bukti untuk memvalidasi hipotesis tersebut. Proses ini diimplementasikan melalui dua prosedur utama yang saling memanggil (rekursif): **FindOut** dan **Monitor**.

Interpreter: FindOut (Goal)



Procedure FINDOUT (GOAL)

If (GOAL can be inferred)

then

set RULE-LIST = list all rules whose action part fulfills GOAL

until (RULE-LIST = empty) or (GOAL inferred) do

MONITOR(first or next rule from RULE-LIST)

delete this rule from RULE-LIST

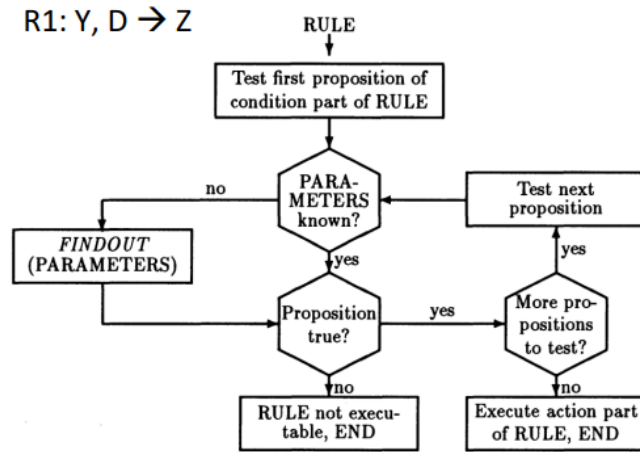
else (request GOAL)

Gambar 7.10: Fungsi FindOut (Goal)

Fungsi ini bertujuan untuk mencari tahu apakah sebuah GOAL (tujuan) bisa dibuktikan benar.

1. Sistem mencari semua aturan di *Production Memory* yang bagian RHS (konklusinya) bisa memenuhi GOAL. Aturan-aturan ini menjadi daftar kandidat.
2. Sistem akan mencoba setiap aturan kandidat satu per satu dengan memanggil prosedur **Monitor**(RULE).
3. Jika tidak ada aturan yang bisa menghasilkan GOAL, sistem akan menganggap GOAL sebagai sebuah sub-tujuan yang harus ditanyakan, entah ke pengguna atau dengan mencarinya langsung di *Working Memory*.

Interpreter: Monitor (Rule)



Procedure MONITOR (RULE)

Test first proposition of condition part of RULE

repeat

If parameters known then

if proposition true then

proposition $\rightarrow$ next

proposition

else RULE not executable

else FINDOUT(PARAMETERS)

Until (no more propositions to test) or (RULE

not executable)

If (no more propositions to test) then

execute action part of RULE

Gambar 7.11: Fungsi Monitor(Rule)

Fungsi ini bertugas memeriksa apakah sebuah RULE (aturan) kandidat dapat dieksekusi.

1. Sistem akan memeriksa setiap kondisi (premis) di bagian LHS aturan tersebut, satu per satu.
2. Untuk setiap premis, sistem akan memeriksa apakah premis tersebut sudah terbukti benar (ada sebagai fakta di *Working Memory*).
3. Jika sebuah premis belum diketahui nilainya, sistem akan memanggil `FindOut(premis_tersebut)`. Di sinilah proses rekursif terjadi, di mana sebuah premis menjadi *sub-goal* baru yang harus dibuktikan.
4. Jika semua premis di LHS berhasil dibuktikan benar, maka RULE tersebut dieksekusi (*fire*), dan konklusinya (RHS) ditambahkan sebagai fakta baru ke *Working Memory*.

7.7 Advantages of RBS

Secara umum, pendekatan RBS memiliki beberapa keunggulan inheren:

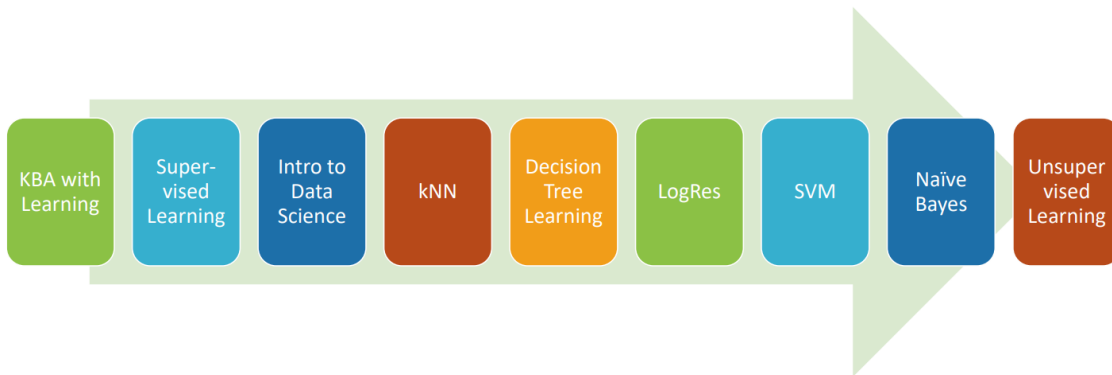
1. **Modularity:** Setiap aturan adalah sebuah potongan pengetahuan yang kecil dan relatif independen. Ini memudahkan pengelolaan basis pengetahuan.
2. **Incrementability:** Aturan baru dapat ditambahkan ke sistem dengan mudah tanpa harus mengubah aturan-aturan lama secara drastis.
3. **Modifiability:** Aturan yang sudah ada dapat diubah atau diperbarui secara independen.
4. **Transparency:** Proses penalaran (terutama pada sistem yang memiliki *explanation component*) bisa dijelaskan kepada pengguna. Sistem bisa menunjukkan aturan-aturan mana yang digunakan untuk mencapai sebuah kesimpulan, sehingga tidak seperti “kotak hitam”.

Bab 8

Learning

8.1	Introduction to Learning	67
8.2	Knowledge-based Agent with Learning	67
8.3	Learning: Why?	68
8.4	Arsitektur KBA with Learning	68
8.5	Design of A Learning Element	69
8.6	Learning Type based on Feedback . .	69

8.1 Introduction to Learning

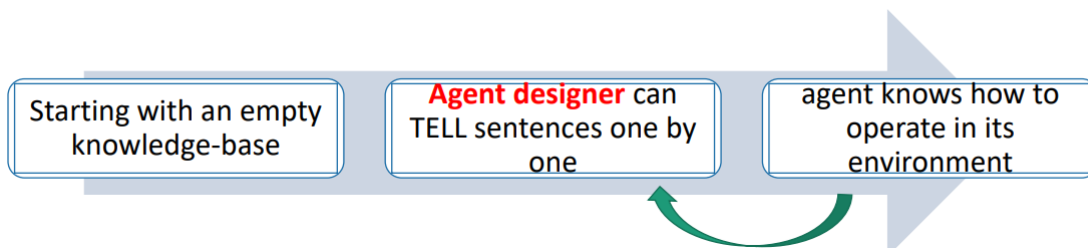


Gambar 8.1: Learning Roadmap

8.2 Knowledge-based Agent with Learning

Konsep

Knowledge-Based Agent (KBA) with Learning adalah sebuah agen cerdas yang tidak hanya beroperasi berdasarkan basis pengetahuan (*knowledge-base*) yang sudah ada, tetapi juga mampu memperbarui dan meningkatkan pengetahuannya secara mandiri melalui observasi dan pengalaman.



Gambar 8.2: Knowledge-based Agent with Learning

Berbeda dengan KBA standar di mana perancang harus memasukkan semua pengetahuan secara manual (melalui perintah TELL), agen pembelajar dapat:

1. **Memulai dari Awal:** Umumnya memulai dengan basis pengetahuan yang kosong atau sangat minim.
2. **Beradaptasi di Lingkungan Asing:** Mampu beroperasi di lingkungan yang awalnya tidak diketahui sama sekali, karena ia dapat belajar tentang dinamika lingkungan tersebut seiring waktu.
3. **Meningkatkan Kinerja:** Secara aktif menggunakan observasi dari dunia untuk memperbaiki performanya dalam menyelesaikan tugas-tugas di masa depan.

Analogi: Bayangkan perbedaan antara robot pabrik yang diprogram untuk satu tugas spesifik (KBA standar) dengan robot penjelajah Mars (KBA with Learning). Robot pabrik akan gagal jika ada sedikit perubahan, sementara robot penjelajah harus bisa belajar dari medan baru, cuaca tak terduga, dan membuat keputusan untuk bertahan hidup dan menyelesaikan misinya.

8.3 Learning: Why?

Kemampuan belajar sangat krusial bagi agen karena tiga alasan utama:

1. **Untuk Menghadapi Lingkungan yang Tidak Diketahui (*Unknown Environments*):**

Perancang sistem tidak mungkin mengetahui semua kemungkinan situasi yang akan dihadapi agen (*designer lacks omniscience*). Kemampuan belajar memungkinkan agen untuk memahami dinamika dunianya sendiri tanpa harus diberi tahu semuanya dari awal.

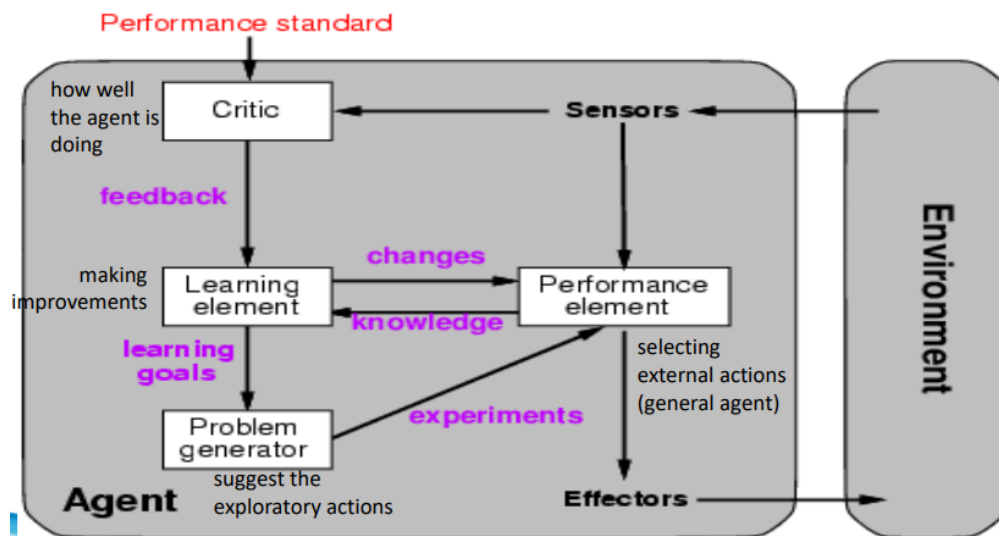
2. **Sebagai Metode Konstruksi Sistem yang Efisien (*System Construction Method*):**

Jauh lebih praktis untuk “melatih” agen dengan menghadapkannya pada data atau contoh dari dunia nyata daripada mencoba menuliskan semua aturan secara manual.

3. **Untuk Meningkatkan Kinerja (*Improve Performance*):**

Agen secara aktif memodifikasi mekanisme pengambilan keputusannya berdasarkan observasi dan umpan balik (*feedback*). Hal ini memungkinkannya menjadi lebih baik dan lebih efisien dalam bertindak di masa depan.

8.4 Arsitektur KBA with Learning

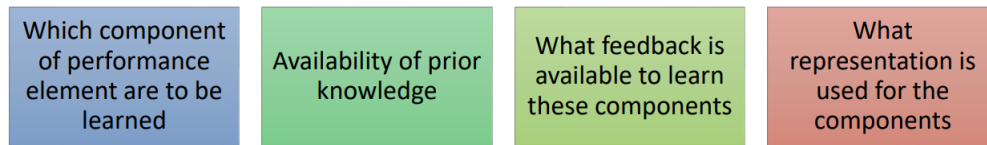


Gambar 8.3: Arsitektur KBA with Learning

Agen pembelajar (learning agent) memiliki empat komponen utama yang saling berinteraksi:

1. **Performance Element:** Komponen utama dari Intelligent Agent, yang menentukan aksi apa yang akan dilakukan oleh aktuator berdasarkan persepsi yang diterima sensor.
2. **Learning Element:** Mengubah pengetahuan terhadap *performance element*. Perubahannya dilakukan berdasarkan *feedback* yang diberikan oleh *critic*.
3. **Critic:** Mengukur aksi yang dilakukan *agent* seberapa baik berdasarkan standar performa tertentu.
4. **Problem Generator:** Menyarankan aksi eksplorasi yang mungkin dibutuhkan oleh *agent* akibat kemungkinan lingkungan yang tidak diketahui

8.5 Design of A Learning Element



Gambar 8.4: Pertimbangan Desain Learning Element

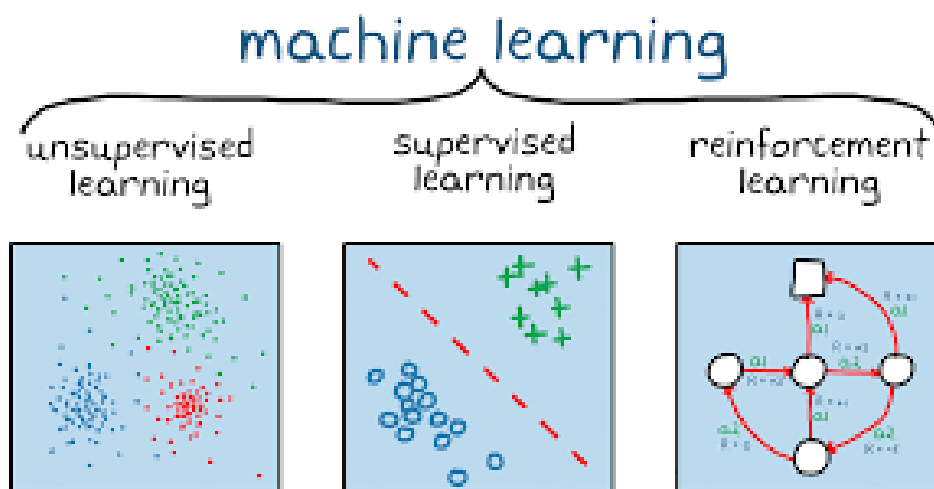
Learning element memberikan perubahan pada sistem sehingga bersifat adaptif, dalam arti *task* yang dilakukan menjadi lebih efisien dan efektif. Dalam mendesain *learning element*, terdapat empat hal yang harus dipertimbangkan:

1. Komponen apa dari *performance element* yang akan diperbaiki kinerjanya
2. Ketersediaan *prior knowledge*
3. *Feedback* yang tersedia untuk proses pembelajarannya
4. Representasi yang digunakan dalam *performance element*

8.6 Learning Type based on Feedback

Terdapat tiga tipe *learning* berdasarkan *feedback* yang diperoleh:

1. **Unsupervised learning:** Ketika tidak ada *feedback*, hanya diberikan kumpulan data tanpa label, lalu akan dicari *cluster* terbaik dari data tersebut.
2. **Supervised learning:** Ketika diberi pasangan *input/output*, lalu mempelajari suatu fungsi yang memetakan dari *input* ke *output*.
3. **Reinforcement learning:** *Agent* akan belajar dari *series reinforcement* yang didapatkan, dapat berupa *reward* atau *punishments* sehingga *Agent* akan menentukan aksi yang dipilih dari hal tersebut.



Gambar 8.5: Tipe *learning*

Bab 9

Supervised Learning

9.1	What is Supervised Learning?	71
9.2	Classification vs Regression	72
9.3	Overfitting vs. Generalization	72
9.4	Supervised Learning Pipeline	73

9.1 What is Supervised Learning?

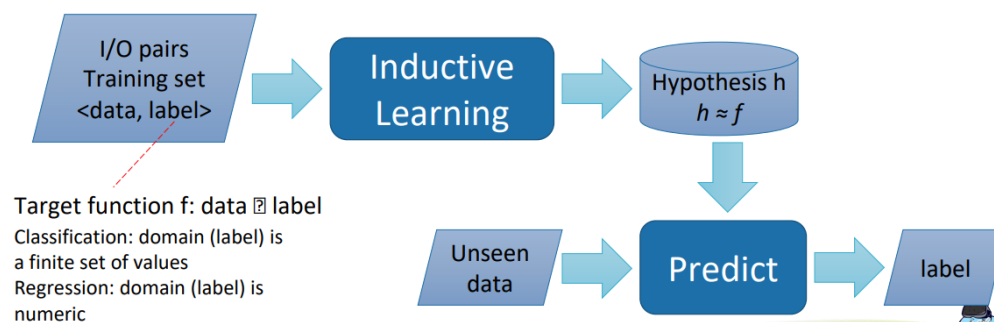
Konsep

Supervised Learning adalah paradigma pembelajaran mesin di mana agen belajar dari sekumpulan data yang sudah diberi **label** dengan benar. Setiap data poin dalam set pelatihan terdiri dari pasangan *input-output*.

Input: Disebut juga fitur, atribut, atau variabel independen (disimbolkan dengan X).

Output: Disebut juga label, target, atau variabel dependen (disimbolkan dengan y atau $f(x)$).

Proses belajar dalam Supervised Learning disebut **Inductive Learning**. Ini adalah proses untuk menyimpulkan sebuah fungsi umum dari contoh-contoh spesifik.



Gambar 9.1: Inductive Learning dalam Supervised Learning

Dalam hal ini:

- Fungsi Target (f): Ini adalah fungsi “ideal” atau “sebenarnya” yang memetakan input ke output. Dalam dunia nyata, kita tidak pernah tahu bentuk pasti dari fungsi ini.
- Hipotesis (h): Ini adalah fungsi yang dipelajari oleh algoritma dari data latih. Tujuannya adalah agar h bisa mengaproksimasi atau mendekati $f(h \approx f)$ sebaik mungkin.
- Training Set: koleksi dari pasangan $\langle data, label \rangle$ ini. Analogi sederhana adalah belajar dengan seorang “guru” yang memberikan soal (*input*) beserta kunci jawabannya (*output*).

Alur prosesnya adalah:

1. Algoritma menerima Training Set.
2. Melalui proses Inductive Learning, algoritma menghasilkan sebuah Hipotesis h .
3. Hipotesis h ini kemudian digunakan untuk membuat prediksi pada data baru yang belum pernah dilihat (*unseen data*).

9.2 Classification vs Regression

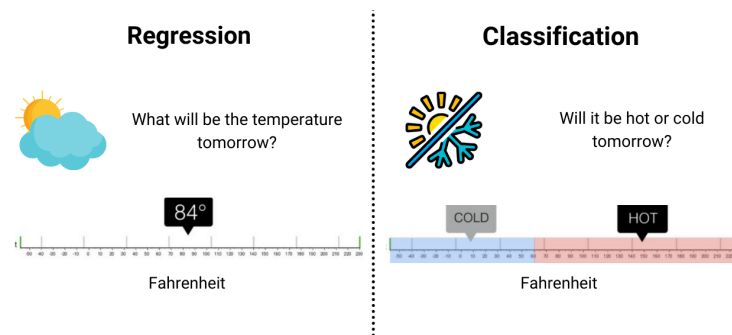
Supervised Learning umumnya dibagi menjadi dua jenis tugas utama, tergantung pada domain dari label/output-nya:

1. Klasifikasi (Classification):

Tujuannya adalah memprediksi label kategori/diskrit. Domain dari labelnya adalah sebuah himpunan nilai yang terbatas. Contoh: Memprediksi apakah sebuah email adalah 'spam' atau 'bukan spam'; mengklasifikasikan spesies bunga ('setosa', 'versicolor', 'virginica').

2. Regresi (Regression):

Tujuannya adalah memprediksi nilai numerik/kontinu. Domain dari labelnya bisa berupa nilai riil apa pun. Contoh: Memprediksi harga rumah berdasarkan luasnya; memprediksi suhu besok hari.



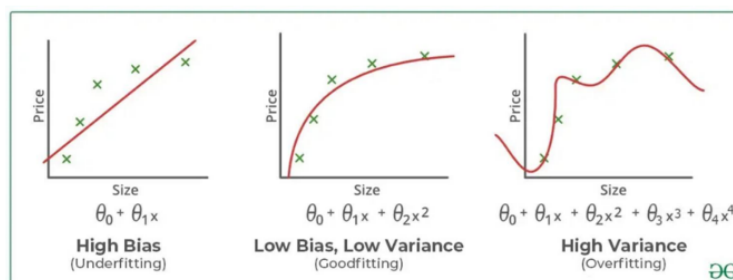
Gambar 9.2: Classification vs Regression

9.3 Overfitting vs. Generalization

Konsep

Model Sederhana (misal: Garis Lurus): Mungkin membuat beberapa kesalahan pada data latih, tetapi cenderung lebih *robust*.

Model Sangat Kompleks: Dapat menciptakan decision boundary yang sangat berliku-liku untuk mengklasifikasikan semua titik di data latih dengan sempurna (akurasi training 100%). Namun, model ini menjadi terlalu “menghafal” data latih, termasuk *noise*-nya. Ini disebut Overfitting.



Gambar 9.3: Bias vs Variance dalam Performa Model

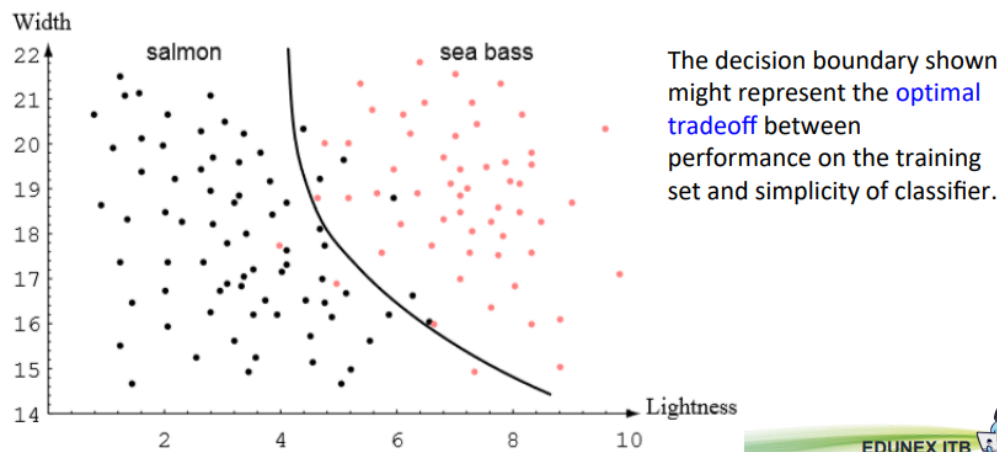
Notes

Konsekuensi *Overfitting*

ketika model ini dihadapkan pada data baru (yang tidak ada dalam set pelatihan), performanya akan sangat buruk. Ia gagal menangkap pola yang mendasari dan hanya menghafal contoh spesifik.

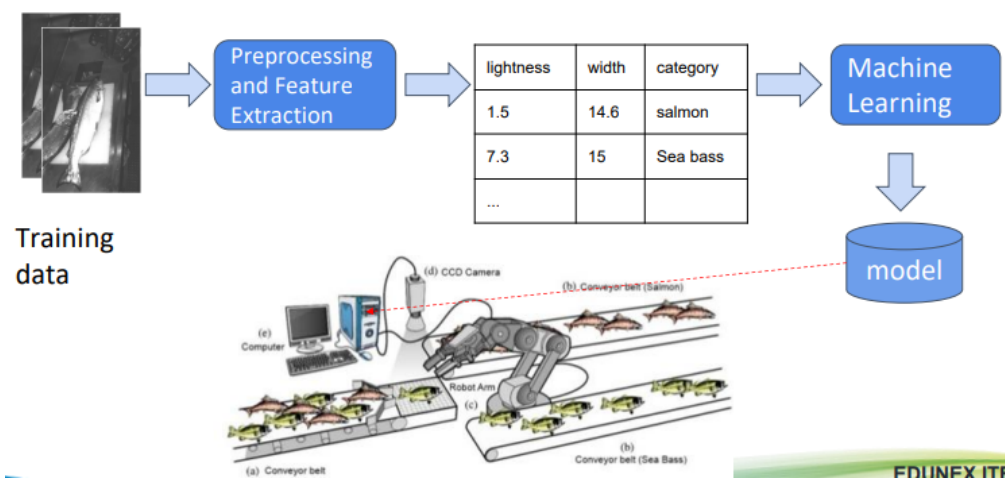
Model Optimal

Model yang baik menemukan keseimbangan. Cukup kompleks untuk menangkap pola yang sebenarnya dalam data, tetapi tidak terlalu kompleks sehingga ia juga menangkap noise. Model seperti ini memiliki kemampuan generalisasi (*generalization*) yang baik, artinya berkinerja baik pada data baru yang belum pernah dilihat.



Gambar 9.4: Workflow Supervised Learning

9.4 Supervised Learning Pipeline



Gambar 9.5: Workflow Supervised Learning